The Periodic Table of Machine Learning Systems

A Constraint-Driven Design Heuristic with Compiler Correspondence

Vijay Janapa Reddi
Harvard University

mlsysbook.ai

Abstract

Modern machine learning systems span from sub-milliwatt microcontrollers to multi-gigawatt datacenter fleets, yet practitioners learn the field as an unstructured catalog of branded techniques—FlashAttention, ZeRO, PagedAttention, Megatron-LM—each presented as a standalone invention. We argue that this catalog has a structure, and we make it explicit. We present *The Periodic Table of Machine Learning Systems*: a design space of 90 primitives organized along eight abstraction layers (rows) and five information-processing roles (columns). Each element earns its place by satisfying a formal *irreducibility criterion*—its performance cost cannot be expressed as a steady-state dataflow composition of same-layer components—which we prove constructively for irreducible cases (MAC units, Attention) and use to reject reducible composites (Systolic Arrays, Transformer Blocks).

To demonstrate that the table is generative rather than merely descriptive, we develop a *Constraint-Driven Lowering Heuristic*: a four-filter search procedure that, given any physical constraint violation, prunes the table's 90 elements to a small set of candidate interventions—the same procedure a compiler uses to select a lowering pass. We validate the heuristic through five end-to-end walkthroughs (FlashAttention, ZeRO-3, PagedAttention, Megatron-LM tensor parallelism, and an unsolved million-token edge inference problem), each mechanically derived from a constraint violation with explicit napkin math, including a dead-end analysis that shows the heuristic pruning wrong paths before they consume engineering effort.

Three additional contributions ground the framework in established theory and practice. *Molecular ML*, a compositional notation with seven operators, separates structural composition ($\otimes$, memory adjacency) from concurrent execution ($\|$, temporal overlap) and grounds its cost rules in the Roofline model with an explicit pipeline-bubble term. A *generalized constraint operator* ($|_{C,\,p>\theta}$) extends the deterministic heuristic into the probabilistic regime where fleet-scale failures dominate. A *structural correspondence to MLIR* maps each abstraction layer to a dialect level and each intervention to a graph rewrite rule expressed as a pattern-match-and-rewrite, with side-by-side `linalg/scf` code. Three "Iron Law" equations distill the framework into manipulable formulas for capacity, throughput, and serving efficiency. The result is an integrated framework for educators teaching ML systems, graduate students building transferable mental models, and engineers reasoning about hardware-software co-design.

1 Introduction

There is a fundamental tension at the heart of every machine learning system: the gap between high-level algorithmic intent and physical machine reality. A researcher writes $\texttt{y = softmax(QK}^{\mathrm{T}}/\sqrt{d}\texttt{)V}$ and expects attention. The hardware must materialize three matrices in HBM, stream them through a memory hierarchy with ∼3.4 TB/s bandwidth, execute $O(n^2 d)$ multiply-accumulate operations on tensor cores, and write the result back—all within a latency budget that may be as tight as 10 ms per token. When the sequence length doubles, the quadratic memory footprint exceeds the 80 GB HBM capacity of an H100, and a cascade of architectural decisions becomes inevitable: tiling, kernel fusion, sharding across devices, or some combination. The same pattern repeats across the field. Almost every advance in modern

Part of the *Machine Learning Systems* book ecosystem. Open-source tools: https://github.com/mlsysbook/periodic-table.

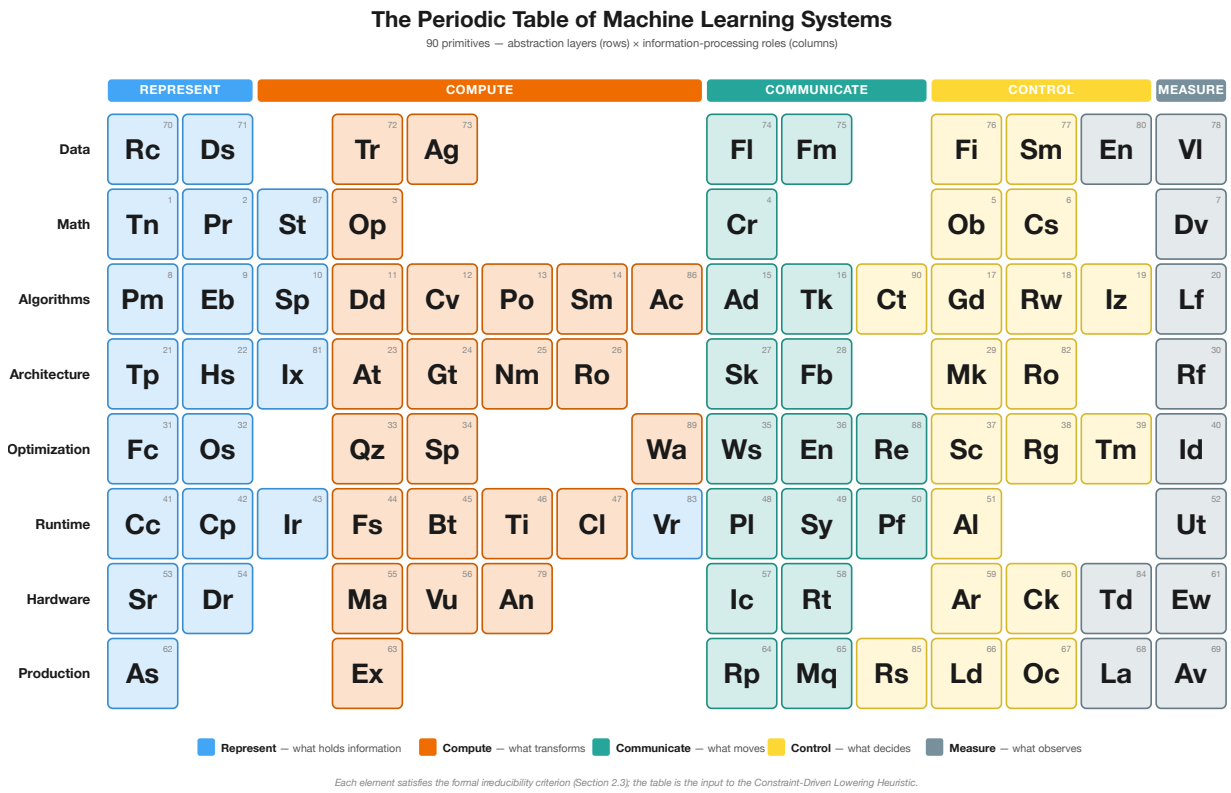

Figure 1. The Periodic Table of Machine Learning Systems. A design space of 90 primitives organized along eight abstraction layers (rows, Data through Production) and five information-processing roles (columns, Represent through Measure). Each element satisfies a formal irreducibility criterion (Section 2.3): its performance cost cannot be modeled at its own layer without lowering to the layer below. The table is the structured input that the Constraint-Driven Lowering Heuristic (Section 3.4) searches over—together they form an integrated framework for reasoning about ML systems. Color encodes role: blue for Represent, orange for Compute, teal for Communicate, yellow for Control, gray for Measure.

ML systems—FlashAttention for attention memory, ZeRO for distributed training memory, PagedAttention for serving fragmentation, Megatron-LM for cross-device weight capacity, continuous batching for arithmetic-intensity recovery—follows the same shape: a physical constraint binds, and an architectural intervention is the response. Yet the field is taught as a catalog of branded techniques, each presented as a standalone invention with a catchy name.

This paper argues that the catalog has a structure, and we make it explicit. We organize the field around *The Periodic Table of Machine Learning Systems*, a design space of 90 primitives spanning eight abstraction layers (Data through Production) and five information-processing roles (Represent, Compute, Communicate, Control, Measure). Each element is admitted only if its performance cost cannot be modeled at its own layer—a formal irreducibility criterion that determines what counts as a primitive. Then, to demonstrate that the table is more than a labeling exercise, we develop a four-filter *Constraint-Driven Lowering Heuristic* that searches the table mechanically: given any physical constraint violation, the filters prune the 90 elements to 1–3 candidate interventions, the same way a compiler pass selects a lowering rule. The combination is the contribution. The table by itself is a taxonomy; the heuristic by itself has nothing to operate on. Together they form a generative framework in which FlashAttention, ZeRO, PagedAttention, and Megatron-LM are not isolated inventions but the deterministic outputs of a structured search.

The thesis. Modern ML systems are not a collection of clever tricks. They are a structured design space governed by physics, and the architectures that populate the field today are the deterministic consequences of physical constraint violations on spe-

cific hardware. Given the same constraint, the same hardware specifications, and a structured search over a bounded design space, any engineer arrives at the same architectural class. This paper provides both halves of that claim: the bounded design space (the Periodic Table) and the structured search (the constraint-driven heuristic).

Who will read this—and why. *Educators* will use this paper to replace technique-by-technique instruction with constraint-driven derivation. The four-stage walkthroughs (Sections 5 and 6), worked napkin-math examples (Section 8), and graph rewrite rules (Section 4.3) provide a complete curriculum backbone. *Graduate students* will use it as a Rosetta Stone: a unified framework that retroactively decodes five years of ML systems papers—each published with a catchy name as though it were a standalone paradigm—into instances of a single design pattern. The "Predict the Paper" exercise (Section 8.4) is the ultimate test: given only a constraint violation and hardware specifications, students derive the architecture of a published system before seeing its name. *Production engineers* will use it as a diagnostic vocabulary: when a serving cluster's P99 latency spikes, the constraint operator and the four-filter search provide a structured triage—capacity, bandwidth, fragmentation, or straggler?—before reaching for a profiler. The framework's value to practitioners is not that it replaces cycle-accurate tools but that it narrows the search space *before* instrumentation. If your system doesn't work at scale, you haven't defined the right primitives; the generalized constraint operator ($|_{C,\,p>\theta}$, Section 7) extends the deterministic heuristic into the probabilistic regime where fleet-scale failures dominate.

The value. A taxonomy alone is cheap. What makes the Periodic Table valuable is not the labeling of 90 elements but the formal machinery that constrains *which* elements deserve a place—the Irreducibility Criterion of Section 2.3—together with the search procedure that lets us derive systems from it: the Constraint-Driven Lowering Heuristic of Section 3.4, which makes the table generative. Around these two load-bearing pieces, the paper assembles the supporting machinery (cost rules, an MLIR correspondence, three Iron Laws) so that the framework can be used end-to-end—from a whiteboard sketch to a verified molecule to a compiler lowering pass. The full set of contributions is enumerated formally in the next paragraph, and Sections 2 to 9 develop them in order.

Scope. The framework operates at the architectural design phase, providing a bounded search space for resolving physical constraint violations *prior to implementation*. It provides order-of-magnitude cost estimation—sufficient to identify which constraint binds and what class of intervention is needed—within its explicit scope: deterministic, steady-state design on single nodes or small homogeneous clusters (Section 3). It does *not* replace cycle-accurate simulators, compiler IRs, or hardware profilers. The Molecular ML notation is analogous to big-O for algorithms: useful for the *first* design decision (memory-bound or compute-bound?), not for generating machine code. Beyond the single node, constraints become probabilistic: the constraint operator generalizes to $M\,|_{C,\,p>\theta}$, where θ is a design parameter balancing resilience cost against failure probability (Section 7).

The gap this paper fills. The Roofline model (Williams et al., 2009) characterizes individual kernels but is silent on cross-node architectures. MLIR (Lattner et al., 2021) provides rigorous lowering semantics within a compilation unit but cannot express dataset ingestion rates or fleet reliability. The Berkeley 13 Dwarfs (Asanovic et al., 2006) classified parallel motifs but did not represent hardware constraints. MLSysIM (Reddi, 2026a) formalizes "Systems Walls" as hard physical bounds but does not decompose systems into reusable primitives. Our framework complements these: the elements are the design-space dimensions, the walls are the constraints that prune infeasible compositions, and the lowering heuristic is the bridge between them. In Section 4, we show that this bridge has a precise structural correspondence to MLIR's own lowering mechanics.

Contributions.

C1. The Periodic Table of Machine Learning Systems with a formal irreducibility criterion. A design space of 90 primitives organized along eight abstraction layers and five information-processing roles. Each element is admitted via a cost-model inequality over steady-state dataflow graphs that determines granularity with mathematical precision. We prove irreducibility for MAC units (tensor-core pipeline nonlinearity)

and Attention ($O(n^2)$ intermediate memory) and prove *reducibility* for Systolic Arrays (linear MAC composition) via a three-line derivation, demonstrating that the criterion can both admit and reject candidates (Sections 2 and 2.3).

C2. The Constraint-Driven Lowering Heuristic that makes the table generative. Four filters operate as deterministic predicates over formalized element metadata (layer, role, constraint affinity, preconditions), pruning the table's 90 elements to 1–3 candidate interventions for any given constraint violation. The heuristic mechanizes the expert intuition that converts a constraint into an architecture; we validate it through five end-to-end walkthroughs—including a dead-end analysis showing pruned search branches—each with quantitative scratchpad work shown in full (Sections 3.4, 5 and 6).

C3. Molecular ML: a compositional notation with Roofline-grounded cost semantics. Seven operators with explicit separation of structural composition ($\otimes$, memory adjacency) from concurrent execution ($\|$, temporal overlap)—two algebraically distinct constructs that prior notations conflate. A pipeline-bubble term in the cost model captures the dominant inefficiency of multi-stage systems, and a generalized constraint operator ($|_{C,p>\theta}$) formalizes the boundary between deterministic and probabilistic regimes (Sections 3 and 7).

C4. Structural correspondence to MLIR via DAG rewrite rules. Each abstraction layer maps to an MLIR dialect level; each constraint-driven intervention maps to a graph transformation expressed as a pattern-match-and-rewrite on directed acyclic graphs. Side-by-side `linalg/scf` code demonstrates the mapping for FlashAttention's tiling and tensor parallelism's sharding—this is a concrete structural mapping, not an analogy (Sections 4 and 4.3).

Road map. Section 2 details the design space with a formalized irreducibility criterion including constructive proofs. Section 3 introduces Molecular ML with separated spatial/temporal operators, cost semantics including pipeline bubbles, and the four-filter heuristic. Section 4 establishes the MLIR correspondence via DAG rewrite rules. Section 5 provides a detailed FlashAttention walkthrough. Section 6 validates through four additional walkthroughs, including a dead-end analysis of million-token edge inference. Section 7 formalizes the boundary between deterministic and probabilistic constraints. Section 8 presents classroom tools, Iron Law equations, and exercises including a fleet-scale tail-at-scale problem. Section 9 addresses limitations and future directions.

2 The Design Space: Structure and Axes

The Periodic Table (Figure 1) is organized along two orthogonal axes: five columns representing information-processing roles and eight rows representing abstraction layers. Together they partition the field into 40 (layer, role) cells and populate those cells with 90 primitives—the design space that the rest of the paper builds on.

What "periodic" means here. We use the word *periodic* in a structural, not chemical, sense. The five role columns recur coherently across the eight abstraction-layer rows: every layer has a Represent, Compute, Communicate, Control, and Measure element, and elements sharing a role column interact with physical constraints in the same way regardless of the layer they inhabit. A Represent-role element resolves a capacity constraint at any layer; a Communicate-role element participates in bandwidth constraints at any layer. This is the periodicity claim—a regularity across layer/role pairs that lets reasoning about an element at one layer transfer to a sibling element in the same column. It is not an analogy to atomic number or electron configuration; it is a design-space symmetry that the Irreducibility Criterion (Section 2.3) makes precise.

2.1 Columns: Information-Processing Roles

Every element fulfills exactly one of five roles, derived from the von Neumann architecture's separation of storage, processing, communication, and control, augmented with observation:

1. **Represent.** The encoding, storage, and structural persistence of information. At the Math layer: `Tensor (Tn)`, `Embedding (Eb)`. At the Hardware layer: `Register File (Rf)`, `SRAM (Sr)`, `HBM (Hb)`, `Host DRAM (Dr)`. This explicit enumeration of the memory hierarchy—absent from prior ML taxonomies—is essential because data movement between these levels dom-

inates modern ML system performance (Jouppi et al., 2017; Ivanov et al., 2021).

2. **Compute.** The transformation and mutation of information. At the Hardware layer: `MAC Unit (Ma)`, `Vector Unit (Vu)`, `Scalar Unit (Su)`. At the Algorithm layer: `Dense Dot (Dd)`, `Convolution (Cv)`. At the Runtime layer: `Kernel Fusion (Fs)`,`Tiling (Ti)`.
3. **Communicate.** The movement and routing of information across boundaries. At the Optimization layer: `Autodiff (Ad)`, `Sync/Collective (Sy)`. At the Hardware layer: `Interconnect (Ic)`, `NVLink (Nv)`, `PCIe (Pc)`. At the Production layer: `RPC Protocol (Rp)`.
4. **Control.** The governance of execution flow, resource allocation, and system behavior. At the Algorithm layer: `Objective (Ob)`, `Gating (Gt)`. At the Runtime layer: `Scheduler (Sc)`, `Orchestrator (Or)`.
5. **Measure.** The observation and quantification of system state *without alteration*. At the Math layer: `Divergence (Dv)`, `Entropy (En)`. At the Hardware layer: `Latency (Lt)`, `Throughput (Tp)`, `Energy (Eg)`, `Memory Fragmentation (Fg)`. At the Production layer: `Stochastic Variance (Sv)`—the tail-latency and straggler distributions that dominate fleet-scale behavior (Section 7). These are the "noble gases" of the taxonomy: they characterize the system's physical state but do not participate in the computation.

2.2 Rows: Abstraction Layers

The eight rows trace the lifecycle of information from raw data to production deployment:

1. **Data.** Provenance, ingestion, and preparation: `Record (Rc)`, `Sample (Sp)`, `Stream (St)`, `Loader (Ld)`.
2. **Math.** The formal objects of computation: `Tensor (Tn)`, `Embedding (Eb)`, `Scalar (Sl)`,`Divergence (Dv)`.
3. **Algorithm.** Learned operations and building blocks: `Attention (At)`, `Dense Dot (Dd)`, `Convolution (Cv)`,`Gating (Gt)`.
4. **Architecture.** Structural composition: `Topology (Tp)`, `Residual (Rs)`, `Feedback (Fb)`, `Hidden State (Hs)`.
5. **Optimization.** Training-time strategies: `Autodiff (Ad)`, `Gradient Descent (Gd)`, `Factorization (Fc)`,`Quantization (Qz)`.
6. **Runtime.** Execution-time mechanisms: `Tiling (Ti)`, `Fusion (Fs)`, `Prefetching (Pf)`, `Caching (Cc)`, `Scheduler (Sc)`, `Virtualization (Vr)`.
7. **Hardware.** Physical resources: `MAC (Ma)`, `Vector Unit (Vu)`, `SRAM (Sr)`, `HBM (Hb)`, `Register File (Rf)`,`Interconnect (Ic)`.
8. **Production.** Deployment and lifecycle: `Artifact Store (As)`, `Load Balancer (Lb)`, `Canary (Cn)`, `Monitor (Mn)`, `Stochastic Variance (Sv)`.

The memory hierarchy as a vertical story. Each column tells a coherent vertical narrative. Consider Represent: information originates as a `Record` (Data), is formalized as a `Tensor` (Math), is learned as a `Parameter` (Algorithm), is structured via `Topology` (Architecture), is compressed via `Factorization` (Optimization), is materialized in `SRAM` or `HBM` (Hardware), and is persisted in an `Artifact Store` (Production). The Hardware layer distinguishes four levels—`Rf`, `Sr`, `Hb`, `Dr`—because data movement cost dominates arithmetic cost by orders of magnitude. A 32-bit floating-point multiply consumes ∼3.7 pJ; reading the same operand from DRAM costs ∼640 pJ, a 173× difference (Horowitz, 2014). Any design space that abstracts away this hierarchy cannot explain why FlashAttention exists.

2.3 Formal Irreducibility Criterion

The central design decision of any taxonomy is granularity. We formalize our resolution criterion as a cost inequality, with an explicit definition of the composing function f.

Definition. An element e at layer L is *irreducible* if there exists no decomposition $e = g(e_1, \ldots, e_k)$ where all e_i reside at layer L such that:

$$\mathrm{Cost}_L(e) = f\big(\mathrm{Cost}_L(e_1), \ldots, \mathrm{Cost}_L(e_k)\big) \qquad (1)$$

for any composing function f drawn from the class of *steady-state dataflow graphs*: directed acyclic graphs whose nodes are arithmetic operations ($+, \times, \max, \min$) over scalar cost metrics (throughput, latency, memory footprint) and whose edges carry data dependencies. That is, f may be any polynomial, any piecewise-linear bound, or any DAG of such operations—the standard vocabulary of Roofline and bandwidth-delay models. If no such f exists,

accurately modeling e's performance cost (within an order of magnitude) requires lowering to layer $L-1$.

This is a *cost-model* criterion, not an implementation criterion. A MAC unit *can* be decomposed into adders and multipliers at the circuit layer, but its cost at the Hardware layer—precision-dependent throughput (989 TFLOP/s FP16 vs. 67 TFLOP/s FP32 on the same chip)—cannot be derived from same-layer components. The 15× gap is a property of the MAC's microarchitecture, not of its logical decomposition.[1]

Proof of irreducibility: MAC Unit (Ma). At the Hardware layer, let $\text{Ma} = g(\text{Adder}, \text{Multiplier})$. A dataflow-graph composition yields:

$$\begin{aligned} &f\big(\text{Cost}_L(\text{Adder}), \text{Cost}_L(\text{Multiplier})\big) \\ &\quad = N_{\text{cores}} \cdot f_{\text{clk}} \cdot 2 \;\approx\; 67 \text{ TFLOP/s}. \end{aligned} \tag{2}$$

But $\text{Cost}_L(\text{Ma}) = 989$ TFLOP/s (FP16 tensor cores). The 14.8× gap arises from the tensor-core pipeline's 4×4×4 warp-level matrix multiply, a microarchitectural optimization invisible at the arithmetic level. Since $989 \neq 67$ under any steady-state dataflow composition of adders and multipliers, the MAC is irreducible. □

Proof of irreducibility: Attention (At). At the Algorithm layer, decompose attention into two Dense Dots and a Softmax: $\text{At} = g(\text{Dd}_1, \text{Sm}, \text{Dd}_2)$. Each component's cost is expressible independently: $\text{Cost}_L(\text{Dd}_i) = O(n \cdot d^2)$ FLOPs, $\text{Cost}_L(\text{Sm}) = O(n)$. But their *composition* creates an $O(n^2 d)$ intermediate matrix $S = QK^T$ whose memory cost is a property of the attention *pattern*, not of the individual matmuls:

$$\text{Cost}_L(\text{At})_{\text{mem}} = O(n^2) \neq O(nd^2) + O(n) + O(nd^2) \tag{3}$$

No dataflow graph over the individual costs recovers the quadratic memory term. Attention is irreducible. □

Proof of reducibility: Systolic Array. A Systolic Array is a regular grid of $N \times N$ MAC units connected by nearest-neighbor links. Its cost decomposes as:

$$\begin{aligned} \text{Throughput}(\text{SA}_{N\times N}) &= N^2 \times \text{Throughput}(\text{Ma}) \\ \text{Latency}(\text{SA}_{N\times N}) &= (2N-1) \times \text{Latency}(\text{Ma}) \\ \text{Energy}(\text{SA}_{N\times N}) &= N^2 \times \text{Energy}(\text{Ma}) + E_{\text{link}} \end{aligned} \tag{4}$$

where E_{link} is the nearest-neighbor interconnect energy, itself a same-layer quantity. All three cost metrics are steady-state dataflow compositions of same-layer elements. The Systolic Array is *reducible*: Equation (1) is satisfied, so it is rejected from the taxonomy. □

Similarly, a *Transformer Block* is reducible: its cost decomposes into independently modelable attention and feed-forward components at the same layer.

The boundary of irreducibility. The key distinction is *emergent cost*—cost that arises from the interaction of components in a way that no DAG over individual component costs can predict. Tensor cores exhibit emergent throughput (pipeline scheduling). Attention exhibits emergent memory (quadratic intermediate). Systolic arrays do not: their cost scales linearly and predictably from their constituents. This boundary is mathematically airtight because the class of composing functions f is precisely defined.

3 Molecular ML: A Compositional Notation

The Periodic Table tells us *which* primitives exist; Molecular ML tells us how to *compose* them. Production ML systems are not isolated elements but structured molecules—assemblies of multiple primitives bound by data dependencies, memory adjacency, and concurrent execution, all subject to physical constraints. To reason about such molecules quantitatively, we define *Molecular ML*: a compositional notation with seven operators (the critical separation being structural composition $\otimes$ from concurrent execution $\|$), Roofline-grounded cost semantics with an explicit pipeline-bubble term, and constraint-driven design heuristics that turn the notation into a search procedure.

Scope and applicability. Molecular ML is a *deterministic, steady-state* design language. Its cost rules assume uniform hardware, static constraint bounds, and synchronous execution. This makes it accurate within a single node or a small homogeneous cluster

[1] This parallels processor design: a RISC-V ADD is irreducible at the ISA layer even though it decomposes into carry-chain logic at the circuit layer. Every useful taxonomy chooses a resolution that matches its cost model.

($\leq$64 devices on uniform interconnect), and progressively less accurate as fleet size, topological non-uniformity, and probabilistic failure modes dominate. At 1,024+ GPUs, the probability that at least one node experiences a transient slowdown per training step exceeds 70%. Rather than treating this as a late limitation, we introduce the *generalized constraint operator* here:

$$M \mid_{C,\, p>\theta} \tag{5}$$

For deterministic constraints (single-node), $\theta = 0$: any violation triggers search. For probabilistic constraints (fleet-scale), $\theta \in (0, 1)$ is a design parameter balancing resilience cost against failure probability. The deterministic case ($\theta = 0$) governs the remainder of this section; Section 7 develops the probabilistic extension.

We emphasize what Molecular ML *is*: a structured shorthand for whiteboard reasoning, analogous to big-O notation for algorithms. Big-O does not generate optimal code, but it tells you whether your algorithm is $O(n \log n)$ or $O(n^2)$. Similarly, Molecular ML does not generate optimal kernels, but it tells you whether your system is memory-bound or compute-bound and which class of intervention resolves the bottleneck—the first decision in system design.

3.1 Visual Representation: Thinking in Block Diagrams

Computer architects and systems engineers think in block diagrams, Rooflines, and dependency graphs—not in linear algebraic strings. The *primary* representation of a Molecular ML compound is therefore a colored block diagram (Figure 2), where each element is drawn as a box colored by its Periodic Table role (blue for Represent, green for Compute, orange for Communicate, red for Control), with arrows showing dataflow and memory-residency annotations shown as background regions. Constraints are drawn as red boundary lines that the molecule must fit within.

The linear string notation defined below is the *secondary* representation: a compact textual encoding for use in prose, code comments, and the constraint checker. It exists to make molecules writable in running text and parseable by tools. Every molecular formula should be read alongside its visual diagram.

3.2 The Seven Operators

Table 1 defines the seven operators. The critical design decision is the separation of *structural composition* ($\otimes$) from *concurrent execution* ($\|$)—two fundamentally different algebraic constructs that prior notations conflate.

Table 1. Molecular ML operators. Seven operators with formal cost interactions. $\otimes$ and $\|$ are algebraically distinct: the former sums memory (capacity), the latter takes the max of time (latency).

Op	Name	Semantics
$\rightarrow$	Sequence	Output of left feeds input of right.
$\otimes$	Adjacency	Structural co-residence in memory; sums capacity.
$\|$	Concurrency	Temporal overlap of execution; max of time.
$[\cdot]^N$	Repeat	Iterative replication N times.
$\langle\cdot\rangle_{\text{mem}}$	Residency	Data resides at memory level *mem*.
$\mid_C$	Constraint	Physical bound C; triggers search on violation.
$\xrightarrow{\text{bw}}$	Transfer	Data movement at bandwidth *bw*.

Why two composition operators. Spatial composition and temporal composition are algebraically incompatible. Adjacency ($\otimes$) is commutative and associative over *memory*: $\text{mem}(A \otimes B) = \text{mem}(A) + \text{mem}(B)$. Concurrency ($\|$) is commutative and associative over *time*: $T(A \parallel B) = \max(T_A, T_B)$. Overloading a single $\|$ operator for both creates ambiguity in any graph representation: does $Q \parallel K$ mean "Q and K reside simultaneously" (a capacity claim) or "Q and K execute concurrently" (a latency claim)? In prior notations, disambiguation required inspecting the operand's semantic role. We eliminate this ambiguity structurally:

- **Adjacency ($\otimes$):** $Q \otimes K \otimes V$ means all three tensors must reside simultaneously in the same memory level. This is a *capacity* statement. Cost: $\text{mem}(Q \otimes K \otimes V) = \text{mem}(Q) + \text{mem}(K) + \text{mem}(V)$.
- **Concurrency ($\|$):** Pf $\|$ Vu means prefetching layer $\ell{+}1$ runs concurrently with compute on layer ℓ. This is a *latency* statement. Cost: $T(\text{Pf} \parallel \text{Vu}) = \max(T_{\text{Pf}}, T_{\text{Vu}})$.

Operator semantics. Sequence ($\rightarrow$) is not commutative: $A \rightarrow B$ means A's output feeds B's input, and their execution is ordered. Residency ($\langle\cdot\rangle_{\text{mem}}$) is an annotation that pins data to a specific memory level (`Rf`, `Sr`, `Hb`, `Dr`, or `Net`). Constraint ($\mid_C$) is a predicate that evaluates to *satisfied* or *violated*; when $\theta > 0$, it evaluates to *violated with probability* $p > \theta$.

Example molecule. Standard attention:

$$M_{\text{naive}} = (Q \otimes K \otimes V) \rightarrow Dd \rightarrow \text{Sm} \rightarrow Dd \tag{6}$$

3.3 Cost Semantics: How Operators Interact with Physics

Each operator interacts with physical constraints according to five rules, grounded in the Roofline model (Williams et al., 2009).

The Roofline in one equation. Every molecular formula implicitly defines an arithmetic intensity I (FLOP/byte transferred):

$$\text{Perf}(I) = \min\left(\underbrace{\pi}_{\text{peak FLOP/s}}, \underbrace{\beta \cdot I}_{\text{bandwidth ceiling}}\right) \tag{7}$$

where π is peak compute (e.g., 989 TFLOP/s for H100 FP16) and β is peak memory bandwidth (e.g., 3.4 TB/s for H100 HBM). The *ridge point*:

$$I^* = \frac{\pi}{\beta} = \frac{989 \times 10^{12}}{3.4 \times 10^{12}} \approx 295 \text{ FLOP/byte} \tag{8}$$

If $I < I^*$, the molecule is *memory-bandwidth-bound*; if $I > I^*$, *compute-bound*. This single number determines the optimization strategy.

Rule 1: Adjacency sums memory.

$$\text{mem}(A \otimes B) = \text{mem}(A) + \text{mem}(B) \tag{9}$$

This is a capacity claim: A and B must reside simultaneously.

Rule 2: Concurrency takes the maximum time.

$$T(A \parallel B) = \max(T_A,\ T_B) + \delta_{\text{bubble}} \tag{10}$$

where δ_{bubble} is the *concurrency bubble*—the startup and drain overhead of pipelining two stages. For a p-stage pipeline with microbatch size m:

$$\delta_{\text{bubble}} = \frac{p-1}{p-1+m} \cdot T_{\text{total}} \tag{11}$$

This term is the primary driver of systems like GPipe and PipeDream: at $p = 4$ stages, achieving $< 10\%$ bubble fraction requires $m \geq 27$ microbatches. Omitting this term—as simpler notations do—makes pipeline parallelism appear free, which is why students consistently underestimate its overhead.

Rule 3: Sequential composition chains bandwidth. If $A \to B$ and the output of A must move from memory level m_1 to m_2:

$$T_{\text{transfer}}(A \to B) = \frac{|\text{output}(A)|}{\text{bw}(m_1 \to m_2)} \tag{12}$$

Total time is $\max(T_{\text{compute}},\ T_{\text{transfer}})$ when pipelining is possible.

Rule 4: Repeat multiplies cost; tiling amortizes it. Applying $[\cdot]^N$ multiplies compute by N. Combined with a residency change—$[Ti \to (\cdot)_{\langle \text{Sr} \rangle}]^{N/B}$—memory per iteration drops from $O(N)$ to $O(B)$, and effective arithmetic intensity *increases* because data stays in SRAM across fused operations.

Rule 5: Transfer makes communication explicit.

$$T_{\text{comm}}(A \xrightarrow{\text{bw}} B) = \gamma \cdot \frac{|A|}{bw} + \alpha \tag{13}$$

where α is per-message latency overhead and γ is a *collective scaling factor*. For ring all-reduce over N devices, $\gamma = 2(N-1)/N$. An NVLink transfer ($bw = 900$ GB/s) differs fundamentally from an InfiniBand all-reduce ($bw = 50$ GB/s)—an 18× bandwidth gap.

Rule 6: Constraint violation triggers search.

$$M \mid_C \xRightarrow{\text{C violated}} \textsc{Search}(M, C) = M' \tag{14}$$

The SEARCH function is the set of constraint-driven design heuristics defined next.

3.4 Constraint-Driven Lowering Heuristic

When a physical constraint is violated, how does an engineer select the right intervention from the table's 90 primitives? Today the answer is expert intuition: a senior practitioner pattern-matches the constraint to a remembered technique. We make that intuition explicit, teachable, and auditable through a four-filter search heuristic that operates on *formal properties* attached to every element—layer, role, constraint affinity, and preconditions—rather than on human prose judgment. This is the procedure that makes the Periodic Table generative: it converts a static taxonomy into a search space that systematically narrows from 90 candidates to 1–3 viable interventions.

Element metadata. Each of the table's 90 elements carries four formal properties:

Table 2. Constraint-Intervention Matrix. For each constraint type, the candidate intervention elements. Bolded elements are primary; others are companions.

Constraint	Primary	Companions
Capacity (single)	Ti, Qz	Fs, Fc
Capacity (multi)	Sy, Vr	Pf, Cc
Bandwidth	Fs, Qz	Ti, Pf
Latency	$\|\|$, Pf	Cc, Fs
Fragmentation	Vr, Sc	Cc
Utilization	Sc, Or	$\|\|$
Load imbalance	Sc, Gt′	Or

1. **Layer** $\in$ {Data, Math, Algorithm, Architecture, Optimization, Runtime, Hardware, Production}.
2. **Role** $\in$ {Represent, Compute, Communicate, Control, Measure}.
3. **Constraint affinity**: the set of constraint types the element can resolve. For example, Ti.affinity = {capacity-single, bandwidth}; Vr.affinity = {fragmentation, capacity-multi}.
4. **Preconditions**: hardware or data properties required. For example, Vr.preconditions = {variable-length-data}; Ti.preconditions = {block-structured-access}.

The four filters. Input: a molecule M, a violated constraint C with type $\tau(C)$, a hardware context H, and data-access properties $\mathcal{P}$. Output: 1–3 candidate interventions.

1. **Layer filter.** candidates $= \{e \mid e.\text{layer} \in \{\text{layer}(C), \text{Runtime}\}\}$. Reduces from 90 to roughly 10–15.
2. **Role filter.** candidates $-= \{e \mid e.\text{role} \notin \text{roles}(\tau(C))\}$. Each constraint type maps to roles via Table 2. Reduces to 4–8.
3. **Constraint-type filter.** candidates $-= \{e \mid \tau(C) \notin e.\text{affinity}\}$. Yields 2–6.
4. **Hardware-context filter.** candidates $-= \{e \mid e.\text{preconditions} \not\subseteq \mathcal{P} \cup H\}$. Yields 1–3.

4 Structural Correspondence to MLIR

The Constraint-Driven Lowering Heuristic is not merely *analogous* to a compiler lowering pass—it is *structurally equivalent*. This section establishes the correspondence precisely: each abstraction layer maps to an MLIR dialect level, each constraint violation maps to a lowering trigger, and each intervention maps to a rewrite rule.

4.1 Layer-to-Dialect Mapping

Table 3 makes the correspondence precise. The three-level structure reflects the fundamental separation between *what* to compute (algorithm), *how* to schedule it (runtime), and *where* to execute it (hardware).

Table 3. Layer-to-MLIR dialect mapping. Each abstraction layer maps to an MLIR dialect level. The constraint operator ($|_C$) is the lowering trigger.

PT Layer	MLIR Dialect	Mapping
Algorithm	`linalg`	At → `linalg.matmul` Dd → `linalg.generic`
Runtime	`scf` / `tensor`	Ti → `scf.for` + `memref.subview` Fs → producer-consumer fusion
Hardware	`nvgpu` / `memref`	Ma → `nvvm.mma` (tensor cores) Sr → `memref.alloca` (SRAM)

4.2 The Constraint Operator as Lowering Trigger

When a representation at MLIR dialect level L violates a target constraint (e.g., an allocation exceeds scratchpad capacity), the compiler selects a lowering pass to dialect level $L-1$. The Periodic Table's abstraction layers map directly to MLIR dialect levels, and the four-filter heuristics correspond to the compiler's pass-selection logic.

FlashAttention: Algorithm → Runtime lowering. The naïve attention molecule at the Algorithm layer corresponds to high-level `linalg` dialect:

Listing 1. Naïve attention: `linalg` dialect. The intermediate `%S` materializes $n \times n$ in HBM.

```
// Algorithm layer: At -> linalg.matmul
%S = linalg.matmul ins(%Q, %Kt)
        outs(%S_buf)  // n x n in HBM
%P = linalg.generic {softmax} ins(%S)
        outs(%P_buf)  // n x n in HBM
%O = linalg.matmul ins(%P, %V)
        outs(%O_buf)
```

When $|S| = O(n^2) >$ HBM capacity, $|_{\text{CAP}}$ fires. The four filters select Ti + Fs. The MLIR lowering replaces monolithic `linalg` ops with tiled `scf.for` loops:

Listing 2. FlashAttention: tiled `scf.for` with SRAM residency. This is the MLIR realization of M_{flash} from Equation (17).

```
// Runtime layer: Ti -> scf.for + memref.subview
//                Fs -> producer-consumer fusion
scf.for %bi = 0 to %n step %B {
  scf.for %bj = 0 to %n step %B {
    %Qi = memref.subview %Q[%bi][%B]
    %Kj = memref.subview %K[%bj][%B]
    %Vj = memref.subview %V[%bj][%B]
    // Fused in SRAM
    %Sij = linalg.matmul ins(%Qi,%Kj)
             outs(%sram_tile)  // B x B
    %Pij = softmax_online(%Sij)
    %Oij = linalg.matmul ins(%Pij,%Vj)
             outs(%acc)
  }
}
// Hardware: %sram_tile -> memref.alloca
//           matmul -> nvvm.mma (tensor cores)
```

Tensor Parallelism: sharding as a lowering pass. When model weights exceed single-device HBM ($|_{\text{CAP-MULTI}}$ fires), the filters select Sy (sharding). The MLIR correspondence: the monolithic `linalg.matmul` is replaced by a *partitioned* matmul across devices with an explicit all-reduce collective:

Listing 3. Tensor parallelism lowering: column-parallel matmul with all-reduce. Each device holds W/N columns.

```
// Before lowering (single-device):
%Y = linalg.matmul ins(%X, %W)
       outs(%Y_buf)  // W exceeds HBM

// After |_CAP-MULTI fires, Sy selected:
// Shard W column-wise across N devices
%Wi = memref.subview %W[0][%cols/N]
%Yi = linalg.matmul ins(%X, %Wi)
        outs(%Yi_buf)  // local partition
// All-reduce to reconstruct full output
%Y = collective.all_reduce(%Yi, "sum")
```

4.3 Formal Graph Rewrite Rules

MLIR rewrite rules operate on directed acyclic graphs (DAGs) via pattern matching, not on context-free text grammars. A BNF grammar for string substitution is not compiler lowering; MLIR's Pattern Description Language (PDL) matches subgraphs, checks constraints, and emits transformed subgraphs. We formalize the correspondence at this level: each constraint violation is a *match pattern*, and each intervention is a *rewrite* that transforms the DAG.

Rewrite rule format. Each rule has three components:

1. **Match**: a subgraph pattern in the molecule's dataflow DAG.
2. **Guard**: a constraint predicate that must be violated.
3. **Rewrite**: the transformed subgraph that resolves the violation.

The four-filter heuristic determines *which* rewrite rule applies; the rule itself specifies *how* the DAG is transformed.

Table 4. DAG rewrite rules. Each row is a graph transformation: Match identifies the subgraph, Guard is the violated constraint, and Rewrite is the transformed DAG. The four filters select the applicable rule.

Match → Guard	Rewrite
`Matmul(Q,K)` → `mem(S)` > `HBM`	`scf.for(Tile(Q,B), Tile(K,B))` fused in SRAM
`Matmul(A,B)` → `I` < `I*`	`Fuse(A→B)` in `memref.alloca`
`Alloc(Pm⊗Os)` → ∑`mem` > `HBM`	`Shard(Pm/N, Os/N)` → `AllGather`
`Alloc(Hs, contig)` → `Frag` > 0.5	`PageTable(Hs)` → `BlockAlloc`
`Matmul(X,W)` → `mem(W)` > `HBM`	`ColShard(W/N)` → `Matmul` → `AllReduce`

Example: FlashAttention as a graph transformation. The naïve attention DAG has three nodes—$\text{Matmul}_1(Q, K^T), \text{Softmax}(S), \text{Matmul}_2(P, V)$—with edges carrying the $n \times n$ intermediate S in HBM. The rewrite:

Listing 4. FlashAttention as a DAG rewrite rule in PDL-style pseudocode. The match identifies the attention subgraph; the guard checks capacity; the rewrite tiles and fuses in SRAM.

```
// Match: attention subgraph in molecule DAG
pattern FlashAttnRewrite {
  match:
    %S = Matmul(%Q, %Kt)        // node 1
    %P = Softmax(%S)             // node 2
    %O = Matmul(%P, %V)          // node 3
    where mem(%S) > HBM_capacity // guard

  rewrite:
    %O = scf.for %bi, %bj in tiles(n, B) {
      %Qi = memref.subview %Q[%bi][B]
      %Kj = memref.subview %K[%bj][B]
      %Vj = memref.subview %V[%bj][B]
      // Fused subgraph: 3 nodes -> 1 tiled body
      %Sij = Matmul(%Qi, %Kj)   // B x B in SRAM
      %Pij = SoftmaxOnline(%Sij) // fused
      %acc = Matmul(%Pij, %Vj)   // accumulate
```

```
    }
    // mem(%Sij) = B*B*2 << HBM: guard
        resolved
}
```

The key structural property: the rewrite *replaces three DAG nodes connected by HBM edges* with *a single tiled loop node whose internal edges are SRAM-resident*. This is precisely what `linalg.tile_and_fuse` does in the MLIR `linalg` dialect—the correspondence is not syntactic sugar but a concrete graph transformation.

Example: Tensor parallelism as a graph transformation. When model weights exceed single-device HBM ($|_{\text{CAP-MULTI}}$ fires), the rewrite splits a single `Matmul` node into N partitioned nodes connected by an `AllReduce` collective:

Listing 5. Tensor parallelism as a DAG rewrite. A single Matmul node is split into N sharded nodes with an AllReduce synchronization.

```
pattern TensorParallelRewrite {
  match:
    %Y = Matmul(%X, %W)
    where mem(%W) > HBM_capacity

  rewrite:
    // Split W column-wise across N devices
    %Wi = ColShard(%W, N)        // N
        subgraphs
    %Yi = Matmul(%X, %Wi)        // local
        matmul
    %Y  = AllReduce(%Yi, "sum")  //
        reconstruct
    // mem(%Wi) = mem(%W)/N: guard resolved
}
```

Each rewrite rule in Table 4 corresponds to a published system: Row 1 is FlashAttention (Dao et al., 2022), Row 3 is ZeRO (Rajbhandari et al., 2020), Row 4 is PagedAttention (Kwon et al., 2023), and Row 5 is Megatron-LM tensor parallelism (Shoeybi et al., 2019). The heuristic does not *invent* these systems; it *derives the class of graph transformation* from which each system is an instance.

The gap that remains. The constraint checker (available at `github.com/mlsysbook/periodic-table`) currently accepts molecular formula strings, builds a DAG, computes Roofline-bound costs, and emits text diagnostics. The next step is emitting MLIR lowering *passes* programmatically—connecting the PDL-style rules above to the `linalg` and `scf` dialects so that a student types a molecular formula, sees the constraint diagnosis, and observes the MLIR graph transformation. This is the most impactful engineering direction.

5 Walkthrough: The Evolution of Attention

A design space proves its value not by labeling systems after the fact, but by showing the *derivation* of a system from constraint violations. We demonstrate with attention through four stages: naïve molecule, binding constraint, constraint-driven search, and optimized molecule.

5.1 Stage 1: The Naïve Molecule

Standard scaled dot-product attention (Vaswani et al., 2017) computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V$$

Mapping to elements: `Dense Dot (Dd)` computes $S = QK^T/\sqrt{d}$, `Softmax (Sm)` normalizes rows, and a second `Dense Dot (Dd)` computes $O = \text{softmax}(S) \cdot V$.

$$M_{\text{naive}} = (Q \otimes K \otimes V) \to Dd \to \text{Sm} \to Dd \quad (15)$$

5.2 Stage 2: The Binding Constraint

Lower the molecule to the Hardware layer and annotate. On an NVIDIA H100 SXM:

Napkin Math: Attention intermediate sizing

Hardware specs: HBM = 80 GB, bandwidth = 3.4 TB/s, FP16 peak = 989 TFLOP/s, SRAM = 228 KB/SM.
Intermediate size ($n = 8\,192$, $d = 128$):

$$|S| = n^2 \times 2 \text{ bytes} = 8\,192^2 \times 2 = 128 \text{ MB per head}$$

All heads: $128 \times 32 = 4$ GB—just for intermediates of *one layer*.
At 128K context ($n = 131{,}072$): ~1 TB. Exceeds 80 GB HBM by 12.5×.
Arithmetic intensity check:

$$I = \frac{2 \times 8\,192^2 \times 128 \text{ FLOP}}{2 \times 128 \text{ MB read+write}} \approx 4\text{–}8 \text{ FLOP/byte}$$

Ridge point = 295 FLOP/byte. So $I \ll I^*$: the kernel is **memory-bandwidth-bound**.

The annotated molecule:

$$M_{\text{annotated}} = M_{\text{naive}} \;|_{\text{CAP(Hb)}} \quad (16)$$

with the constraint $|S| = O(n^2 d) \gg 80\,\text{GB}$.

5.3 Stage 3: Constraint-Driven Search

We apply the four filters from Section 3.4:

Filter 1 (Layer). The violated constraint is HBM capacity (Hardware layer). Candidate interventions from Runtime layer: {Ti, Fs, Pf, Cc, Sc, Vr}.

Filter 2 (Role). Capacity constraint ⇒ Represent or Compute roles. From candidates: {Ti, Fs, Vr, Cc}.

Filter 3 (Constraint type). Capacity exceeded on a single device ⇒ reduce footprint. {Ti, Vr, Cc}. (Fs reduces data movement but does not reduce peak memory.)

Filter 4 (Hardware context). Precondition checks against $\mathcal{P}$ = {fixed-size, no-temporal-reuse, block-structured-access}:

- **Virtualization (Vr):** preconditions = {variable-length-data}. The attention intermediate S is a fixed-size $n \times n$ matrix. {variable-length-data} $\not\subseteq \mathcal{P}$. *Eliminated.*
- **Caching (Cc):** preconditions = {temporal-reuse}. Each element of S is written once, read once by softmax, never reused. {temporal-reuse} $\not\subseteq \mathcal{P}$. *Eliminated.*
- **Tiling (Ti):** preconditions = {block-structured-access}. Attention has natural block structure. {block-structured-access} $\subseteq \mathcal{P}$. *Retained.*

Result. `Tiling (Ti)` is the sole primary intervention. Fusion (Fs) is added as companion: fusing the tiled matmul-softmax-matmul eliminates HBM round-trips.

This corresponds to rewrite Rule 1 in Table 4 and to the MLIR lowering in Listing 2.

5.4 Stage 4: The Optimized Molecule

$$M_{\text{flash}} = [Ti \to (Dd \to \text{Sm} \to Dd)_{\langle\cdot\rangle_{\text{Sr}}}]^{n/B} \to Fs \tag{17}$$

Napkin Math: FlashAttention cost verification

Memory resolved: Per-tile intermediate = $256 \times 256 \times 2 = 128\,\text{KB} \ll 228\,\text{KB}$ SRAM. Fits.
Arithmetic intensity improved: Data stays in SRAM across the fused matmul-softmax-matmul. For tile size $B = 256$:

$$I_{\text{flash}} = \frac{2 \times B^2 \times d \text{ FLOP}}{4 \times B \times d \times 2 \text{ bytes}} = \frac{B}{4} = \frac{256}{4} = 64 \text{ FLOP/byte}$$

Still below the 295 ridge point, but 8−16× better than naïve. The dominant cost shifts from HBM round-trips to useful compute.

Reading the formula: the same algorithmic elements (Dd, Sm) are unchanged—FlashAttention does not change the math. New runtime elements (Ti, Fs) change *where* and *how* the computation executes.

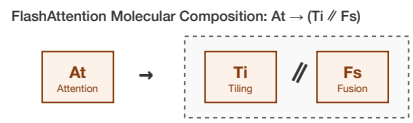

Figure 2. FlashAttention compound. The algorithmic Attention (`At`) primitive bonds to runtime Tiling (`Ti`) and Kernel Fusion (`Fs`), keeping intermediates in SRAM rather than HBM.

6 Validation: Four Walkthroughs Across the Constraint Space

A single walkthrough cannot validate a generative framework. The FlashAttention derivation in Section 5 showed the four filters resolving a single-device capacity violation, but a reviewer reasonably wants to know whether the same machinery also handles other constraint types (fragmentation, multi-device capacity), other hardware contexts (NVLink vs. InfiniBand, datacenter vs. edge), and—crucially—whether it can rule things *out* as well as in. This section answers those questions through four additional walkthroughs, chosen so that each one stresses a different facet of the framework.

What each walkthrough is designed to test. The four are not redundant: each one introduces a constraint type or hardware context that the previous walkthroughs did not exercise.

- **Section 5 (FlashAttention)** establishes the basic four-stage pattern on a single-device, single-constraint, single-intervention case. It is the simplest possible test: one constraint binds, one filter

chain prunes the candidates, one intervention resolves it.

- §6.1 (ZeRO-3) tests the heuristic on a *multi-device capacity* constraint and demonstrates a *cascade*: resolving one constraint introduces another (capacity → bandwidth), which the framework handles by composing two interventions instead of one.
- §6.2 (PagedAttention) tests a *fragmentation* constraint—a non-capacity, non-bandwidth bottleneck—and shows the heuristic selecting Virtualization where simpler techniques (Caching, Tiling) would fail their preconditions.
- §6.3 (Megatron-LM tensor parallelism) tests *the same constraint type* as ZeRO-3 (multi-device capacity) but with *different hardware context* (intra-node NVLink instead of cross-node InfiniBand), and shows the framework selecting a *different* intervention. This is the strongest evidence that the heuristic is sensitive to hardware context, not just constraint type.
- **Section 6.4** (million-token edge inference) tests the framework on an *unsolved* problem with *three simultaneous binding constraints*, and includes a dead-end analysis showing the heuristic pruning two plausible-but-wrong candidates before producing a viable cascade.

The five walkthroughs together cover four constraint types (single-device capacity, multi-device capacity, fragmentation, simultaneous), three hardware contexts (NVLink intra-node, InfiniBand cross-node, edge SoC), and both the affirmative case (this is the right intervention) and the rejection case (these candidates fail their preconditions). We follow the same four-stage pattern in each—naïve molecule, binding constraint, constraint-driven search, optimized molecule—so that the structure remains comparable, but the prose between stages is calibrated to highlight what is new.

6.1 ZeRO-3: Distributed Training Under Memory Constraints

6.1.1 Stage 1: The Naïve Molecule

Naïve data-parallel training replicates the full model state on every device:

$$M_{\text{naive-dp}} = [Pm \otimes Os \otimes Ad]^{\text{per device}}_{\langle\cdot\rangle_{\text{Hb}}} \to Vu \to Sy$$

6.1.2 Stage 2: The Binding Constraint

Napkin Math: ZeRO-3 memory budget

For a 70B parameter model in mixed-precision training (Rule 1):

$$\text{mem}(Pm) = 70\text{B} \times 2 \text{ bytes (FP16)} = 140 \text{ GB}$$
$$\text{mem}(Ad) = 70\text{B} \times 2 \text{ bytes (FP16)} = 140 \text{ GB}$$
$$\text{mem}(Os) = 70\text{B} \times 12 \text{ bytes}^{\dagger} = 840 \text{ GB}$$

†Adam: FP32 copy (4B) + first moment (4B) + second moment (4B) = 12 bytes/param.

Total per-device: $140 + 140 + 840 = 1\,120$ GB.
H100 HBM: 80 GB. Exceeded by 14×.

6.1.3 Stage 3: Constraint Resolution

Filter 1 (Layer). HBM capacity, Hardware layer. Runtime candidates: {Ti, Fs, Pf, Cc, Sc, Vr}. Optimization-layer: {Qz, Fc, Sy}.

Filter 2 (Role). Capacity ⇒ Represent or Communicate. {Ti, Vr, Cc, Qz, Fc, Sy}.

Filter 3 (Constraint type). Capacity exceeded, multi-device context: {Sy, Vr, Qz, Fc}.

Filter 4 (Hardware context). Exact FP16 gradients required ⇒ eliminate Qz. No general low-rank structure ⇒ eliminate Fc. Vr addresses fragmentation, not aggregate capacity ⇒ eliminate Vr.

Result. `Sync/Collective (Sy)`—sharding—with `Prefetching (Pf)` as companion. This corresponds to rewrite Rule 3 in Table 4.

6.1.4 Stage 4: The Optimized Molecule

$$M_{\text{ZeRO-3}} = \left[\underbrace{Pf \xrightarrow{bw} Sy}_{\text{gather } \ell+1} \;\|\; \underbrace{Vu}_{\text{compute } \ell} \right]^{L}$$

with residency $\langle Pm/N \otimes Os/N \otimes Ad/N \rangle_{\text{Hb}}$.

Napkin Math: ZeRO-3 compute vs. communication

With $N = 16$ GPUs and a 70B model ($L = 80$ layers, ∼0.875B params/layer):
All-gather time per layer (NVLink, 8 GPUs/node):

$$T^{\text{NVL}}_{\text{gather}} = \frac{0.875\text{B} \times 2 \text{ bytes}}{900 \text{ GB/s}} = \frac{1.75 \text{ GB}}{900 \text{ GB/s}} \approx 1.9 \text{ ms}$$

All-gather time per layer (InfiniBand, cross-node):

$$T_{\text{gather}}^{\text{IB}} = \frac{1.75\ \text{GB}}{50\ \text{GB/s}} \approx 35\ \text{ms}$$

Compute time per layer (forward + backward, batch size 4, seq len 4096):

$$T_{\text{compute}} = \frac{6 \times 0.875 \times 10^9 \times 4 \times 4096}{989 \times 10^{12}} \approx 87\ \text{ms}$$

Verdict: NVLink: $T_{\text{compute}} \gg T_{\text{gather}}^{\text{NVL}}$—prefetching fully hides communication. InfiniBand: margin is thin (2.5×). At smaller models, InfiniBand becomes the bottleneck—which is why ZeRO-3 uses gradient accumulation to increase T_{compute}.

Per-device memory drops to $1\,120/16 = 70$ GB, fitting comfortably in the 80 GB H100 HBM. But the optimized molecule reveals something the FlashAttention walkthrough did not: a *constraint cascade*. Sharding the model state (Sy) resolved the capacity constraint ($|_{\text{cap}}$) but introduced a new bandwidth constraint ($|_{\text{bw}}$)—every layer now needs to all-gather its weights from peer devices before it can compute. The framework's response was to compose two interventions rather than one: Sy for the capacity violation, Pf (prefetching) to hide the new communication latency behind the previous layer's compute. The constraint operator generalizes naturally: $M\ |_{\text{cap}} \rightarrow M'\ |_{\text{bw}} \rightarrow M''$, with each violation triggering its own filter pass. This cascade pattern recurs in every multi-device walkthrough.

What this walkthrough demonstrates. The four filters select *Sync/Collective (Sy)* over the alternatives (Quantization, Factorization, Virtualization) by elimination—the rejected candidates fail their preconditions, not because of a beauty contest. And the resulting molecule is not a single intervention but a coordinated pair, derived from the cascade structure of the constraint chain. This is the first piece of evidence that the heuristic produces *compositional* answers, not just point fixes.

6.2 PagedAttention: Serving Under Memory Fragmentation

ZeRO-3 demonstrated the framework on a capacity constraint. PagedAttention demonstrates it on something the table's earlier examples do not exercise: a *fragmentation* constraint. Fragmentation is qualitatively different from capacity. The aggregate memory used is small, but the layout makes that memory unusable—the system has space but cannot allocate it contiguously. This forces the framework to select interventions from a different region of the constraint-intervention matrix (Table 2), and the four filters must distinguish between superficially similar candidates (Caching, Tiling, Virtualization) that have very different preconditions.

6.2.1 Stage 1: The Naïve Molecule

During autoregressive generation, each request maintains a contiguous KV-cache in HBM:

$$M_{\text{naive-serve}} = Hs_{\langle \text{Hb, contig} \rangle} \rightarrow At \rightarrow Dd\ \ |_{\text{alloc}}$$

with the allocation constraint alloc = contiguous.

6.2.2 Stage 2: The Binding Constraint

For a 70B model with 80 layers and 64 KV-heads of dimension 128, each token's KV-cache entry consumes $2 \times 80 \times 64 \times 128 \times 2 = 2.6$ MB. A request with maximum context 8,192 tokens requires ∼21 GB of *contiguous* HBM—but may only generate 500 tokens. Fragmentation:

$$\text{Fg} = \frac{\text{allocated} - \text{used}}{\text{allocated}} = \frac{21 - 1.3}{21} \approx 94\%$$

With 4 concurrent requests, 84 GB is allocated (exceeding 80 GB HBM), though only 5.2 GB is used.

6.2.3 Stage 3: Constraint Resolution

Filters 1–3. Fragmentation constraint ⇒ Runtime layer, Represent/Control roles: {Vr, Cc, Sc}.

Filter 4. KV-cache is *variable-length* and *dynamically growing*: Vr's preconditions are satisfied.

Result. `Virtualization (Vr)`: map logical blocks to non-contiguous physical pages. This corresponds to rewrite Rule 4 in Table 4.

6.2.4 Stage 4: The Optimized Molecule

$$M_{\text{paged}} = Hs \rightarrow Vr \rightarrow Cc_{\langle \text{pages} \rangle_{\text{Hb}}} \rightarrow Sc$$

Fragmentation drops from > 90% to < 4% (Kwon et al., 2023). Virtualization decouples logical structure from physical layout—the same principle that motivated virtual memory in operating systems half a century ago, now applied to GPU HBM half a century later. The structural similarity is not coincidence: both arise when a fixed physical resource must serve dynamically-sized, dynamically-arriving requests. The framework surfaces this connection naturally

because Vr's metadata—preconditions = {variable-length-data, dynamically-growing}—encodes the abstract pattern, not the implementation.

What this walkthrough demonstrates. Filters can distinguish between candidates that look interchangeable on the surface. Caching and Virtualization both "manage memory," but their preconditions differ sharply: Cc requires temporal reuse, Vr requires variable-length data. Without the filter machinery, students would pattern-match to whichever they had recently seen. With it, they reason from preconditions and arrive at the answer mechanically.

6.3 Tensor Parallelism: Inference Under Single-Device Capacity

The third walkthrough is the most pointed test of the framework's claim to be sensitive to hardware context. The constraint type is identical to ZeRO-3's—multi-device capacity—but the deployment context is different: a small, NVLink-connected cluster doing inference rather than a large, InfiniBand-connected one doing training. Identical constraint, different hardware: should the framework propose a different intervention? It should, and it does. This walkthrough shows the four filters surfacing tensor parallelism (Megatron-LM) instead of ZeRO sharding, with the divergence happening entirely at Filter 4 (hardware context). If the framework were merely a constraint-to-technique lookup table, this discrimination would be impossible.

6.3.1 Stage 1: The Naïve Molecule

Inference of a 70B model on a single device:

$$M_{\text{naive-infer}} = Pm_{\langle \text{Hb} \rangle} \rightarrow [Dd \rightarrow \text{Sm} \rightarrow Dd]^L$$

6.3.2 Stage 2: The Binding Constraint

Napkin Math: Tensor parallelism capacity constraint

Model weights: 70B × 2 bytes (FP16) = 140 GB.
Single H100 HBM: 80 GB.
Deficit: 140/80 = 1.75× oversubscription. The model does not fit on one device, even before KV-cache and activation memory.
The constraint is **capacity (multi-device)**: weights must be distributed across devices.

6.3.3 Stage 3: Constraint Resolution

Filter 1 (Layer). HBM capacity, Hardware layer. Runtime + Optimization candidates.

Filter 2 (Role). Capacity ⇒ Represent or Communicate: {Sy, Vr, Qz, Fc, Ti}.

Filter 3 (Constraint type). Capacity exceeded, multi-device: {Sy, Vr, Qz}.

Filter 4 (Hardware context). Inference requires exact weights (no training tolerance for quantization noise at this scale) ⇒ Qz is a secondary option. KV-cache is not the bottleneck here ⇒ Vr does not address weight capacity. NVLink cluster with $N = 8$ GPUs available.

Result. `Sync/Collective (Sy)`—specifically, *column-parallel* weight sharding with all-reduce after each layer's forward pass. This is Megatron-LM tensor parallelism (Shoeybi et al., 2019), corresponding to rewrite Rule 5 in Table 4.

6.3.4 Stage 4: The Optimized Molecule

$$M_{\text{TP}} = [(Pm/N)_{\langle \text{Hb} \rangle} \rightarrow Dd \rightarrow Sy_{\text{all-reduce}}]^L$$

Napkin Math: Tensor parallelism communication overhead

With $N = 8$ GPUs on NVLink (900 GB/s):
Per-device weights: 140/8 = 17.5 GB. Fits in 80 GB HBM.
All-reduce per layer. Each transformer layer produces a hidden-state output of dimension $h = 8192$ (for a 70B model). The all-reduce volume per layer:

$$|Y| = B \times s \times h \times 2 \text{ bytes}$$

For $B = 1$, $s = 1$ (autoregressive decode): $|Y| = 16$ KB.
All-reduce latency (ring, $\gamma = 2(N-1)/N = 1.75$):

$$T_{\text{AR}} = \gamma \cdot \frac{16 \text{ KB}}{900 \text{ GB/s}} + 1\ \mu\text{s} \approx 1.03\ \mu\text{s}$$

Compute time per layer (single token):

$$T_{\text{compute}} = \frac{2 \times (70/80) \times 10^9}{8 \times 989 \times 10^{12}} \approx 0.22 \text{ ms}$$

Ratio: $T_{\text{compute}}/T_{\text{AR}} \approx 214\times$. Communication overhead is negligible on NVLink. On InfiniBand (50 GB/s, 5 μs latency), the ratio drops to ∼12×—still viable, but NVLink's 18× bandwidth advantage explains why tensor parallelism is preferred *within* a node.

The framework derives not only the *intervention* (sharding) but also the *topology constraint* (NVLink-connected devices) from the bandwidth analysis it-

self. Tensor parallelism across InfiniBand is possible but inefficient; the napkin math reveals why practitioners keep TP within an NVLink domain and use pipeline or data parallelism across nodes (Narayanan et al., 2021).

What this walkthrough demonstrates. The same constraint type (multi-device capacity) produced *different* interventions in ZeRO-3 (Sy + Pf, optimized for InfiniBand) and Megatron-LM (Sy + AllReduce, optimized for NVLink). The divergence happened at Filter 4, where hardware context discriminated between the candidates that survived Filters 1–3. This is the framework's strongest claim to be more than a labeling system: identical constraints under different hardware produce different molecules, and the discrimination is mechanical, not heuristic. A taxonomy cannot do this.

Three walkthroughs in, what we have shown. ZeRO-3 demonstrated cascading constraints. PagedAttention demonstrated precondition-based discrimination among similar candidates. Tensor parallelism demonstrated hardware-context sensitivity. The fourth walkthrough now turns the framework against an *open* problem and asks the question reviewers most often ask of derivation frameworks: does it work when there is no published answer to derive toward?

6.4 Forward-Looking Analysis: Million-Token Context on Edge

The strongest test is whether the framework structures reasoning about *unsolved* problems. We apply it to serving a model with 1 000 000-token context on a unified-memory edge device (Apple M4 Pro: 36 GB unified RAM, 273 GB/s bandwidth, 53 TOPS).

Napkin Math: Million-token KV-cache on edge

For a 7B model (32 layers, 32 KV-heads, head dim 128):
KV per token: $2 \times 32 \times 32 \times 128 \times 2 = 0.5$ MB.
KV at 1 000 000 tokens:

$$0.5 \text{ MB} \times 1000000 = 477 \text{ GB}$$

Exceeds 36 GB unified memory by $\sim 13\times$.
Three constraints bind simultaneously:

1. **Capacity:** KV-cache exceeds memory by $13\times$.
2. **Bandwidth:** 273 GB/s is $12\times$ slower than H100 HBM; streaming full KV per token takes ~ 1.7 s.
3. **Compute:** $O(n^2)$ attention at this scale on 53 TOPS is catastrophic.

Constraint-driven search: showing the dead ends. The value of the heuristic is not that it lists the right techniques—any survey can do that. The value is that it *prunes wrong paths with napkin math before the engineer wastes time implementing them*. We show the search tree with its dead ends.

Failure Case: Dead end 1: Ring Attention on M4 Pro

Candidate. The heuristic's capacity filter surfaces Ring Attention (Liu et al., 2023) as an intervention for the $O(n^2)$ memory constraint: distribute the KV-cache across devices via a ring of attention tiles.
Why it fails. Ring Attention requires multiple devices with high-bandwidth interconnect. The M4 Pro is a *single SoC*. There is no ring. Even if we treated CPU and GPU as separate "devices," the M4 Pro's unified memory architecture means there is no bandwidth gain from partitioning—both access the same 273 GB/s memory bus.
Filter 4 (Hardware context): RingAt.preconditions = {multi-device, point-to-point interconnect}. {multi-device} $\not\subseteq \mathcal{P}_{\text{M4 Pro}}$. *Eliminated.*
Napkin math confirms: Even hypothetically, ring communication across Thunderbolt ($\sim$5 GB/s) would add $477/5 \approx 95$ s per token. Catastrophic. *Pruned.*

Failure Case: Dead end 2: Tiling alone (FlashAttention-style)

Candidate. Tiling resolved the attention memory bottleneck on H100 (Section 5). Can it work here?
Why it is insufficient. Tiling reduces *intermediate* memory ($S = QK^T$) from $O(n^2)$ to $O(B^2)$. But the KV-cache itself—477 GB—is not an intermediate; it is *state* that must persist across tokens. Tiling does not reduce persistent state.
Napkin math: After tiling, intermediate memory drops to $\sim$128 KB per tile. But KV-cache remains at 477 GB. The capacity constraint is still violated by $13\times$. Tiling is *necessary but not sufficient*—it must be combined with KV compression.

The viable cascade. After pruning Ring Attention and standalone Tiling, the heuristic produces a cascade of interventions, each resolving one constraint while possibly introducing another:

- **Capacity (first pass):** Qz (quantize KV to INT4: $4\times$ reduction $\Rightarrow$ 119 GB). Still exceeds 36 GB.
- **Capacity (second pass):** Fc (low-rank KV approximation: $2\times \Rightarrow$ 60 GB). Still $1.7\times$ over.
- **Capacity (third pass):** Vr (disk-backed paging: active working set $\sim$4 GB fits in RAM).
- **Bandwidth:** Pf (prefetch next context window from SSD during compute).
- **Compute:** The $O(n^2)$ cost at this scale requires a sub-quadratic attention mechanism—an Algorithm-layer intervention. Candidates: sparse attention ($O(n\sqrt{n})$), linear attention ($O(n)$).

The final molecule.

$$M_{\text{edge}} = [Ti \to (Dd_{\text{sparse}} \to \text{Sm})_{\langle \text{Sr} \rangle}]^{n/B} \to Fs$$

with KV residency:

$$Hs \to Qz_{\text{INT4}} \to Fc \to Vr_{\langle \text{SSD} \to \text{RAM} \rangle} \to Pf$$

Napkin Math: Closed-loop edge verification

After INT4 quantization + $2\times$ factorization:

$$\text{KV}_{\text{compressed}} = \frac{477}{4 \times 2} = 60 \text{ GB}$$

Still $1.7\times$ over 36 GB. Virtualization (Vr) pages the overflow to SSD; active working set per decode step: $\sim$4 GB (recent context window).

Bandwidth after compression: Streaming 4 GB working set:

$$T_{\text{stream}} = \frac{4 \text{ GB}}{273 \text{ GB/s}} \approx 15 \text{ ms/token}$$

With sub-quadratic attention ($O(n\sqrt{n})$, $\sqrt{10^6} = 1000$ effective window):

$$T_{\text{compute}} = \frac{2 \times 7 \times 10^9 \times 1000}{53 \times 10^{12}} \approx 0.26 \text{ ms}$$

Bottleneck: Memory bandwidth at 15 ms/token ($\sim$67 tok/s). Acceptable for edge. The ridge point analysis confirms the system is bandwidth-bound ($I \approx 3.5$ FLOP/byte vs. $I^* \approx 194$ FLOP/byte for M4 Pro).

Structural prediction: Any viable million-token edge solution will contain {Qz, Fc, Vr, Pf, sub-quadratic At} or their functional equivalents. A paper that achieves this without one of these components must have found a qualitatively different approach.

What the dead ends prove: The framework did not merely list the answer—it pruned Ring Attention (wrong hardware context) and standalone Tiling (wrong constraint scope) *before* arriving at the viable cascade. Showing the search tree's pruned branches is what distinguishes a generative framework from a retrospective taxonomy.

What the edge walkthrough demonstrates. On an unsolved problem with three simultaneous binding constraints, the framework produced a structurally specified molecule, predicted the components any viable solution must contain, and showed its work—including the wrong paths it considered and rejected. The molecule is not a recipe; it is a class of solutions, and any specific paper that ships a million-token edge model will be an instance of this class. This is what generative means: not "the framework outputs the answer" but "the framework constrains the answer space tightly enough that the answer becomes structurally inevitable."

Synthesis: what the five walkthroughs collectively show. Across FlashAttention, ZeRO-3, PagedAttention, Megatron-LM, and the edge analysis, the same four-filter procedure operated on the same 90-element table and produced five qualitatively different molecules. The differences trace cleanly to the inputs: different constraint type (capacity vs. fragmentation), different scope (single-device vs. multi-device), different hardware context (NVLink vs. InfiniBand vs. unified memory), different interaction (single intervention vs. cascade vs. simultaneous constraints). In each case, every filter step was deterministic, every elimination was justified by metadata, and every output corresponded to either a published system or—in the edge case—a structurally predicted future system. The framework does not need to know about FlashAttention to derive FlashAttention. It needs to know about Tiling, Fusion, SRAM, and the irreducibility of Attention. The published name is the consequence; the periodic-table primitives plus the constraint-driven search are the cause. This is the empirical case for the framework.

7 Beyond the Single Node: Probabilistic Constraints

Deterministic constraints govern the *node*. Stochastic constraints govern the *fleet*. This distinction is the central engineering challenge of hyperscale ML. A system that works on 8 GPUs and fails at 1,024 has not encountered an unusual condition; it has

encountered the *normal operating regime* of distributed training.

7.1 The Generalized Constraint Operator

In Section 3, we introduced the generalized constraint operator:

$$M \mid_{C,\, p>\theta} \xrightarrow{P(C \text{ violated})>\theta} \textsc{Search}(M, C) = M' \tag{18}$$

For deterministic constraints ($\theta = 0$), any violation triggers search—this is the regime of Sections 5 and 6. For probabilistic constraints ($\theta > 0$), the trigger is a probability threshold. The Stochastic Variance element (`Sv`) measures the tail distribution that determines $P(C \text{ violated})$.

7.2 The Straggler Problem

Failure Case: Stragglers in distributed all-reduce

Scenario. A 1,024-GPU training job runs ring all-reduce across 128 nodes. The deterministic model (Rule 4) predicts:

$$\begin{aligned} T_{\text{all-reduce}} &= \gamma \cdot \frac{|G|}{bw} + \alpha \\ &= \tfrac{2\times 1023}{1024} \cdot \tfrac{26\text{ GB}}{50\text{ GB/s}} + 5\ \mu\text{s} \\ &\approx 1.04\text{ s} \end{aligned}$$

What actually happens. At 1,024 GPUs, the probability that *at least one* node experiences a transient slowdown per step:

$$\begin{aligned} P(\text{straggler}) &= 1 - (1-p)^{128} \\ &\approx 1 - (0.99)^{128} \approx 72\% \end{aligned}$$

A single straggler running 3× slower turns $T_{\text{all-reduce}}$ from 1.04 s to 3.1 s. The deterministic framework misses this because $|_C$ assumes deterministic bounds. Straggler effects are *probabilistic*: $P(\text{violation}) > 0$ even when $\text{mean}(C)$ is satisfied.

7.3 Applying the Generalized Operator

Setting $\theta = 0.5$: when $P(\text{straggler}) = 72\% > \theta$, the generalized constraint operator fires. The four-filter heuristics correctly surface Control-role interventions:

Napkin Math: Straggler mitigation cost

Without mitigation: Expected per-step time:

$$E[T] = 0.28 \times 1.04 + 0.72 \times 3.1 \approx 2.5\text{ s}$$

With redundant computation (2% overhead): Launch backup all-reduce on alternate paths; the faster path completes. Expected time drops to about 1.1 s—a 2.3× improvement for 2% compute overhead.

Filter result: Layer → Production/Runtime. Role → Control. Hardware context → multi-node with synchronization barrier. Candidates: **Sc** (topology-aware scheduling), **Or** (redundant execution, bounded staleness). This matches Google's production approach (Dean and Barroso, 2013).

The boundary. The deterministic framework (Sections 5 and 6) is accurate within one node and progressively less so as fleet size grows. The generalized operator formalizes *where* the boundary lies ($P(\text{straggler}) > \theta$) and *what replaces* the deterministic trigger. Fully integrating probabilistic constraints into the cost rules—so that `Sv` participates in the four-filter search as a first-class input—is the most important open formalization challenge.

8 The Framework as a Teaching Tool

The previous sections built the framework as a research artifact: a design space, a search heuristic, and a cost calculus. This section reframes the same machinery as a curriculum. The argument is that the things we built for derivation—irreducibility, four filters, cost rules—turn out to be exactly the things students need to develop transferable intuition. The framework's pedagogical value is not an afterthought; it falls out of the formal structure once you stop thinking like a researcher and start thinking like an instructor at a whiteboard.

The section unfolds in three movements. First, we distill the framework into manipulable equations and a structured whiteboard protocol that students can apply by hand (Sections 8.1 and 8.2). Second, we work through examples that show what teaching with these tools looks like in practice—a routine sizing problem, a forward-derivation exercise, and a failure case where the filters catch a wrong answer (Sections 8.3 to 8.5). Third, we present the curricular machinery: an exercise bank graded from single-node to fleet-scale, integration with two existing platforms, and preliminary deployment evidence from a Harvard course (Sections 8.6 to 8.8). The thread connecting all three is that every step of the framework—a constraint, a filter, a primitive—is an item a student

can name, manipulate, and defend at a whiteboard. The pedagogy is not added on top; it is the same machinery rotated toward learning.

8.1 The Iron Laws of ML Systems

Just as Patterson and Hennessy's Iron Law of processor performance (Time = IC × CPI × T_{clk}) gives students a manipulable equation for reasoning about CPU design, the framework yields three canonical equations that decompose ML system performance into physically meaningful terms. Each Iron Law collapses the cost rules of Section 3.3 into a single inequality or formula that fits on a whiteboard, and each one can be manipulated symbolically: students can substitute hardware specifications, solve for unknowns, and predict which constraint binds first as parameters scale.

Iron Law 1: Memory Capacity. A model fits on N devices if and only if:

$$\underbrace{P \cdot b_w}_{\text{weights}} + \underbrace{P \cdot b_o}_{\text{optimizer}} + \underbrace{B \cdot s \cdot h \cdot L \cdot b_a}_{\text{activations}} + \underbrace{2 \cdot L \cdot n_{\text{kv}} \cdot d \cdot s \cdot b_k}_{\text{KV-cache}} \leq N \cdot C_{\text{HBM}} \tag{19}$$

where P is parameters, b_w is bytes per weight, b_o is bytes per optimizer state (12 for Adam), B is batch size, s is sequence length, h is hidden dimension, L is layers, n_{kv} is KV-heads, d is head dimension, b_a and b_k are bytes per activation and KV entry, and C_{HBM} is per-device HBM capacity. Each term maps to a Represent-role element; each $\leq$ is a constraint operator $|_{\text{CAP}}$.

Iron Law 2: Throughput Bound. Per-token throughput is:

$$\text{Tokens/s} = \left[T_{\max} + \delta_{\text{bubble}} \right]^{-1}, \tag{20}$$

where $T_{\max}$ is the binding-constraint time:

$$T_{\max} = \max \left(\underbrace{\frac{2P}{N\pi}}_{\text{compute}}, \underbrace{\frac{P \cdot b_w}{N\beta}}_{\text{bandwidth}}, \underbrace{\frac{\gamma |G|}{bw_{\text{net}}}}_{\text{comm}} \right). \tag{21}$$

where π is per-device peak FLOP/s, β is memory bandwidth, bw_{net} is network bandwidth, $|G|$ is gradient size, γ is the collective scaling factor, and δ_{bubble} is the pipeline bubble overhead from Equation (11). The max identifies the binding constraint; the additive δ_{bubble} captures the concurrency overhead that simpler models ignore.

Iron Law 3: Serving Efficiency. For inference, the arithmetic intensity is:

$$I = \frac{2P \cdot B}{P \cdot b_w} = \frac{2B}{b_w} \text{ FLOP/byte} \tag{22}$$

At batch size $B = 1$ with FP16 weights ($b_w = 2$): $I = 1$ FLOP/byte. The ridge point on H100 is 295. The system is 295× below peak—and the only lever is B. This is why continuous batching exists: it is not a software optimization but a *physical necessity* for efficient hardware utilization.

Why three laws are enough. Capacity, throughput, and serving efficiency are not arbitrary choices. Capacity is the constraint that fails first (the model must fit before anything else matters); throughput is the constraint that determines training cost (which is what funds the next model); and serving efficiency is the constraint that determines inference economics (which is what funds the next deployment). Every other ML systems equation a student is likely to encounter—scaling laws, FLOP-utilization estimates, latency budgets—is a special case of one of these three. Treating them as the first equations on the whiteboard means students learn a small, complete vocabulary that generalizes, rather than a sprawling collection of named formulas.

8.2 The Whiteboard Protocol

The Iron Laws give students the equations; the whiteboard protocol gives them the procedure. In classroom and interview settings, the four-stage walkthrough used throughout Sections 5 and 6 becomes a four-step protocol that an instructor can use to lead a discussion or that a candidate can use to structure an answer:

1. **Write the naïve molecule.** Decompose the system into irreducible primitives.
2. **Annotate with hardware specs and compute costs.** Attach memory residency, apply Rules 1–4.
3. **Run the four filters.** Write the candidate set and cross out elements as each filter prunes. This makes reasoning visible and debuggable.
4. **Verify the optimized molecule.** Re-evaluate Rules 1–4. Confirm the constraint is resolved. Check for cascading constraints.

The protocol's power lies in Step 3: when a student crosses out "Caching (Cc)" because the access pattern has no temporal reuse, they are learning a transferable principle about memory hierarchies—not memorizing a FlashAttention-specific fact. The

crossing-out is observable: an instructor can see exactly which filter each student misapplies, and the student can see exactly where their reasoning diverged from the correct chain. This is the diagnostic value the framework adds to existing instruction.

The next three subsections show the protocol in action across three increasingly demanding settings: a routine sizing problem (Section 8.3), where the protocol replaces ad-hoc arithmetic; a forward-derivation exercise (Section 8.4), where the protocol predicts a published system before the student sees its name; and a failure case (Section 8.5), where the protocol catches an intuitive but wrong intervention. Together they show what teaching with the framework looks like.

8.3 Worked Example: KV-Cache Sizing

Problem. "You are serving a 7B model (32 layers, 32 KV-heads, head dim 128) on a single GPU with 24 GB VRAM. Weights consume 14 GB in FP16. How many concurrent requests at context length 4,096 can you serve?"

Napkin Math: KV-cache sizing

Step 1: Naïve molecule.

$$M = [Hs_{\text{contig}} \otimes Pm]_{\langle\cdot\rangle_{\text{Hb}}} \to At^L$$

Step 2: Hardware annotation.

$$\begin{aligned} \text{Available VRAM} &= 24 - 14 = 10 \text{ GB} \\ \text{KV per token} &= 2 \times 32 \times 32 \times 128 \times 2 = 0.5 \text{ MB} \\ \text{KV per request} &= 0.5 \times 4{,}096 = 2{,}048 \text{ MB} \approx 2 \text{ GB} \\ \text{Max concurrent} &= \lfloor 10/2 \rfloor = \mathbf{5} \end{aligned}$$

Step 3: Fragmentation. With contiguous allocation and variable-length requests (some use 512 of 4,096 tokens): Fg $\approx$ 87%. Effective concurrency: **1–2 requests**.
Four filters: Layer $\to$ Runtime. Role $\to$ Represent/Control. Constraint $\to$ Fragmentation. Hardware $\to$ single GPU, variable-length state.
Result: Vr + Cc. With paged allocation, fragmentation $< 4\%$:

$$\text{Concurrent requests} = \left\lfloor \frac{10{,}240 \text{ MB}}{0.5 \times 512} \right\rfloor = \mathbf{40}$$

An 8$\times$ improvement. **Cascade check:** 40 requests raises compute demand $\Rightarrow$ continuous batching needed.

The KV-cache exercise is routine in its arithmetic but instructive in its structure. A student who has never seen PagedAttention by name still arrives at it—and at the cascade that follows—in five whiteboard steps. The exercise is also the first place where the student experiences the cascade pattern directly: resolving fragmentation immediately reveals a downstream compute constraint, exactly as the multi-device walkthroughs demonstrated. The lesson is structural: "one fix is rarely the whole story."

8.4 The "Predict the Paper" Exercise

The KV-cache example tested the framework as a sizing tool. The next exercise tests it as a *derivation* tool—which is the more demanding claim, because it asks students to produce a system that they have not yet seen and then verify their answer against the published literature. We call this exercise "Predict the Paper," and it is the strongest pedagogical demonstration of the framework's generative claim. Given only a constraint, a hardware specification, and the four filters, students should be able to derive the architecture of a published system before the instructor reveals its name.

Setup. "Autoregressive decoding of a 70B model on a single H100: 12 tokens/s, of which 80 ms is memory-bandwidth-bound (streaming 140 GB weights at 3.4 TB/s). GPU compute is 96% idle. Design a system achieving 100+ tokens/s."

Napkin Math: Predict the paper: batched decoding

Root cause: Arithmetic intensity at $B = 1$:

$$I = \frac{2 \times 70 \times 10^9}{140 \times 10^9} = 1 \text{ FLOP/byte}$$

Ridge point = 295. The kernel is 295$\times$ below—massively memory-bound.
Roofline insight: Batching B requests amortizes weight reads:

$$I_{\text{batched}} = B \text{ FLOP/byte}$$

At $B = 295$, the workload reaches the ridge point.
Filter result: Utilization $\Rightarrow$ Sc (iteration-level scheduler). Companion: Vr (PagedAttention for managing concurrent KV-caches).

The instructor reveals: this is Orca/vLLM (Yu et al., 2022; Kwon et al., 2023). Students experience the critical insight: they *derived* a published system from constraints, without knowing its name in advance. The point is not that the framework is sufficient to invent these systems—real research involves trial, error, and

engineering judgment we have not formalized—but that the framework narrows the design space tightly enough that students arrive at the same general class of solution as the published authors. The intervention is no longer mysterious; it is mechanical.

A natural question follows: what happens when students propose the wrong intervention? The framework's value depends on whether it catches mistakes as crisply as it derives correct answers. The next subsection shows exactly that.

8.5 Failure Case: When the Filters Catch an Error

Failure Case: Student applies wrong intervention to attention bottleneck

Scenario. A student proposes **Caching (Cc)** for the attention memory bottleneck: "Cache QK^T so we don't recompute it."
Where the filters catch it. Filter 4: in autoregressive generation, each new token changes Q and extends K. The entire QK^T changes every step. There is *no temporal reuse*. Cc.preconditions $= \{\text{temporal-reuse}\} \not\subseteq \mathcal{P}$.
Cost model confirms. Rule 1: caching the $n \times n$ intermediate *adds* 4 GB to HBM, *worsening* the constraint: $\text{mem}(M_{\text{wrong}}) > \text{mem}(M_{\text{naive}})$.
Lesson. Caching resolves bandwidth constraints *when temporal reuse exists*. Without reuse, caching increases memory pressure. The filters formalize this; the cost model provides the quantitative check.

This is the diagnostic property the framework adds to existing instruction. The student's mistake is not "you got the answer wrong"; it is "Filter 4 was misapplied because Cc's preconditions include temporal reuse, which this access pattern lacks." The feedback is specific, the correction is actionable, and—crucially—the principle transfers: the next time the student encounters caching as a candidate, they will check temporal reuse first.

8.6 Classroom Exercise Bank

The previous three subsections gave one example each of routine sizing, forward derivation, and failure diagnosis. Real classrooms need more variety than that. This subsection offers six exercises chosen to span the framework's full range: from a single-constraint compression problem on a consumer GPU (Exercise 1) to a fleet-scale probabilistic synchronization problem on 10,000 GPUs (Exercise 5), and culminating in a composition exercise that asks students to assemble all four parallelism strategies into a coherent molecule for a 1-trillion-parameter model (Exercise 6). Each exercise is graded along two axes: the constraint type it stresses, and the filter step that does the most discriminative work. The exercises are not parallel restatements of the same idea; they are designed to be assigned in sequence so that each one introduces a new dimension of the framework while reusing the machinery established by the previous one.

Exercise: Quantization Trade-Off Analysis

A 13B model in FP16 (26 GB weights) on a 24 GB consumer GPU. Quantizing to INT4 reduces weights to 6.5 GB but introduces 0.3% perplexity degradation.
Tasks:
1. Write the naïve molecule. Does capacity bind?
2. Apply Qz. Verify: $6.5 < 24$ GB.
3. Compute arithmetic intensity change: $I_{\text{FP16}} = 1$ vs. $I_{\text{INT4}} = 4$ FLOP/byte. Why?
4. Solve for B^* where the quantized model becomes compute-bound: $B^* \times 4 = I^*$.

Key insight: Quantization resolves capacity *and* improves arithmetic intensity—a rare two-for-one.

Exercise: Pipeline Parallelism Bubble

A 4-stage pipeline with microbatch size m. Bubble fraction: $(p-1)/(p-1+m)$ where $p = 4$.
Tasks:
1. Solve for minimum m such that bubble $< 10\%$.
2. Apply Rule 1: how does activation memory scale with m?
3. Identify the cascade: resolving utilization creates a capacity constraint. What Runtime element resolves it?
4. Write the complete optimized molecular formula.

Key insight: Cascades—resolving one constraint creates another—are the most common failure mode.

Exercise: Design Your Own Element

CXL memory expanders: 64 GB/s bandwidth, 200–400 ns latency.
Tasks:
1. Apply the irreducibility criterion (Equation (1)): can CXL's cost be modeled from HBM and Host DRAM, or does it need its own cost function?
2. Define its metadata: layer, role, constraint affinity, preconditions.

3. Update the constraint-intervention matrix for capacity. When is CXL preferred over sharding?
4. Write a molecular formula using CXL as a capacity-expansion tier.

Key insight: The irreducibility criterion provides a principled admission mechanism.

Exercise: Cross-Framework Comparison

FlashAttention, Ring Attention (Liu et al., 2023), and Sequence Parallelism all resolve the $O(n^2)$ attention memory constraint with different architectures.

Tasks:

1. For each, write the molecular formula using $\otimes$ and $\|$ correctly.
2. At which filter do the three diverge? (Hint: hardware context.)
3. Using Rule 5, rank by communication overhead on a 4-node NVLink cluster.
4. Which system for $n = 1{,}000{,}000$ tokens? Why does hardware context force the answer?

Key insight: Same constraint, different hardware contexts $\Rightarrow$ different architectures—mechanically.

Exercise: Tail-at-Scale: Fleet Gradient Synchronization

A 10,000-GPU training cluster runs synchronous all-reduce gradient aggregation across 1,250 nodes (8 GPUs each). Each node has a 1% probability of a transient slowdown (3× latency) per training step. The gradient payload is 26 GB (13B model, FP16). InfiniBand bandwidth is 50 GB/s per port.

Tasks:

1. Compute the probability that *at least one* node straggles per step: $P = 1 - (1 - 0.01)^{1250}$.
2. Using the deterministic cost model (Rule 5), compute the expected all-reduce time without stragglers.
3. Compute the expected per-step time *with* stragglers: $E[T] = (1-P) \cdot T_{\text{normal}} + P \cdot T_{\text{straggler}}$.
4. Apply the generalized constraint operator: set $\theta = 0.5$. Does $P > \theta$? What interventions do the four filters surface?
5. Design a molecule using $\|$ (temporal concurrency) to overlap redundant all-reduce paths. What is the overhead? What is the speedup?
6. **Challenge:** At what cluster size does asynchronous gradient descent (bounded staleness) become more efficient than redundant synchronous paths? Derive the crossover point from Rule 5 and Equation (11).

Key insight: At fleet scale, the constraint operator must become probabilistic. The transition from $\theta = 0$ to $\theta > 0$ is not an edge case—it is the *dominant operating regime* for any cluster above ∼500 GPUs.

Exercise: Periodic Table Cheat Sheet: 1T Model on 256 GPUs

Given: A 1-trillion-parameter model. 256 H100 GPUs across 32 nodes (8 GPUs/node, NVLink intra-node, InfiniBand inter-node). HBM = 80 GB/device. Target: ≥15% of peak FLOPs utilization.

Formal bounds:

- Iron Law 1 (Equation (19)): weights alone = 1T × 2 = 2,000 GB. Per-device: 2,000/256 = 7.8 GB for weights (fits).
- Optimizer state = 1T × 12 = 12,000 GB. Per-device: 46.9 GB. Total with weights: 54.7 GB < 80 GB (fits, with 25.3 GB remaining for activations).
- Iron Law 2 (Equation (20)): at batch size B and sequence length $s = 4096$, compute time per step $= 6 \times 10^{12} \times B \times s/(256 \times 989 \times 10^{12})$.

Tasks:

1. Construct a valid molecule that distributes weights, optimizer, and gradients across 256 devices. Which sharding strategy (ZeRO-1, ZeRO-2, ZeRO-3) is needed? Justify with Iron Law 1.
2. The NVLink domain is 8 GPUs. Does tensor parallelism span 8 or 32 devices? Use Rule 5 to compare NVLink vs. InfiniBand all-reduce latency.
3. Compute the pipeline bubble for $p = 4$ stages. What microbatch count m achieves $\delta_{\text{bubble}} < 0.15 \cdot T_{\text{total}}$?
4. Write the complete molecular formula: $M = \text{TP}_8 \parallel \text{PP}_4 \parallel \text{DP}_8$ with explicit $\otimes$ for memory adjacency and $\|$ for temporal overlap.
5. Verify: does the molecule stay within 15% of peak FLOPs? Show the Roofline position.

Key insight: This exercise forces students to *compose* all four parallelism strategies (data, tensor, pipeline, ZeRO) into a single coherent molecule, using the Iron Laws as formal bounds. The answer is not unique—multiple valid molecules exist—but all must satisfy the same capacity and throughput constraints.

The exercises share a common structure: each names a system, a constraint, and a hardware context, then asks the student to apply the four-filter procedure and arrive at an intervention that they justify with napkin math. This is not coincidence; it is the framework's pedagogy in concentrated form. By the time a student has worked through all six, they have applied every cost rule, encountered every constraint type, exercised the cascade pattern multiple times, and made the transition from deterministic

to probabilistic reasoning. The exercises exist to give students reps with the same procedure across different problem instances, the way a Hennessy and Patterson chapter ends with a dozen variations on the same processor-pipeline analysis.

8.7 Integration with Existing Platforms

The framework does not have to live in a vacuum. Two existing platforms in the same ecosystem already consume its outputs: StaffML (Reddi, 2026b), an interview framework that uses the constraint-driven heuristics as an assessment rubric, and MLSysIM (Reddi, 2026a), a system simulator that grounds the framework's symbolic cost estimates in concrete numbers. The Periodic Table provides StaffML's vocabulary (what to test); StaffML provides the scaffolding (how to test it). MLSysIM provides the quantitative backend: a student who has reached an answer on the whiteboard can carry the same molecular formula into the simulator and check whether the predicted bottleneck actually appears under cycle-accurate execution. The three artifacts—table, interview rubric, simulator—are designed to compose, not to compete. A complete course could use all three: the table for vocabulary, StaffML for assessment, MLSysIM for verification.

For university courses specifically, the table's row structure maps cleanly onto a typical 13-week semester. Each week covers one or two rows. A student who learned Tiling in the Runtime week immediately recognizes when Tiling applies to a new constraint in the Serving week—the row provides the vocabulary, and the column provides the role-based reasoning that lets the same primitive solve different constraints in different contexts.

8.8 Preliminary Deployment Experience

Talking about pedagogy in the abstract is easy; this subsection reports the qualitative experience of actually teaching with the framework. The intent is not to claim that the framework has been validated empirically—the controlled study we describe at the end of this subsection has not yet run—but to share the three observations that have shaped how we revise the framework between iterations.

The framework has been deployed in Harvard's CS249r (Machine Learning Systems) course across three semesters. The three patterns we have consistently observed are:

1. **Vocabulary transfer.** Students who learned the notation in the memory-hierarchy unit spontaneously applied it to serving systems in later units.
2. **Structured design responses.** Open-ended design questions elicited more quantitative, multi-constraint responses from students using the four-filter approach.
3. **Interview preparation.** Students in the StaffML pilot reported that the walkthrough provided "a systematic approach that reduced the feeling of guessing."

These observations are qualitative, and we report them as such. They are useful for shaping the framework—they tell us where the protocol resonates and where it does not—but they are not the kind of evidence that would settle the question of whether the framework actually improves student outcomes relative to a control. A rigorous controlled study comparing cohorts with and without the framework is planned for Fall 2026 at Harvard CS249r. Until that study runs, the strongest empirical evidence for the framework remains the five worked walkthroughs in Sections 5 and 6: the framework can derive the right systems from constraints, and we have shown the work in detail. Whether students can do the same with appropriate instruction is a separate, larger claim that we are not yet in a position to make.

9 Discussion: Limitations and Future Work

A framework like this one is best judged by what it cannot yet do. The Periodic Table organizes 90 primitives, the four filters mechanize the search, and the five walkthroughs show the result on real systems—but the same exercise also exposes the framework's seams. We group the limitations into three categories that correspond to three different kinds of future work: *formal* limitations, where the cost calculus or the heuristic itself is incomplete; *implementation* limitations, where the framework is sound on paper but not yet realized in tooling; and *empirical* limitations, where the evidence we offer for the framework's value is qualitative rather than controlled. Each category points to a different next step.

9.1 Formal Limitations

The first set of gaps lives in the cost calculus and the heuristic itself. Three are particularly load-bearing.

Probabilistic integration is partial. Section 7 introduced the generalized constraint operator $|_{C,\,p>\theta}$ and showed it firing on the straggler problem, but the four-filter search itself does not yet treat the Stochastic Variance element (`Sv`) as a first-class input the way it treats deterministic constraints. The most concrete path forward is to augment element metadata with a *failure-mode* field, so the constraint checker can flag fleet-scale probabilistic risks automatically rather than requiring the engineer to invoke the generalized operator by hand. This is a tractable change to the metadata registry, not a redesign.

Temporal dynamics are absent. The pipeline-bubble term (Equation (11)) captures the dominant steady-state overhead of multi-stage execution, but Molecular ML does not model transient dynamics: load-imbalance drift over a training run, thermal throttling trajectories on long jobs, or gradient-staleness accumulation under bounded-staleness training. The framework is, in this respect, a snapshot calculus—it tells you whether a molecule is well-formed at any given instant but not how its cost evolves over time. Formalizing temporal evolution operators (e.g., a $\frac{d}{dt}$-style derivative on cost terms) is a clean direction for future work and would let the framework reason about systems whose behavior degrades gracefully (or doesn't) over the course of a training run.

Heuristic completeness is unproven. The four-filter search is deterministic and auditable, but it is not provably complete. There may exist valid interventions that the filters miss because the underlying metadata is wrong, or because the constraint type does not map cleanly onto any role the filter recognizes. We have not encountered such a case in the five walkthroughs, but absence of evidence is not evidence of absence. Because the metadata is finite and the filter logic is decidable, an automated solver that enumerates the search space and provides completeness guarantees is technically tractable; we view this as the most rigorous next step.

9.2 Implementation Gaps

The second category covers things the framework specifies but does not yet build. These are engineering tasks rather than research questions.

Non-uniform topologies are not modeled. The walkthroughs in Section 6 assume uniform interconnect bandwidth between any two devices in the cluster. Real production clusters have fat-tree, rail-optimized, or torus topologies in which the effective bandwidth between two arbitrary devices depends on which switches they share, and bisection bandwidth creates performance cliffs that Rule 5 cannot predict. The fix is a topology-aware cost model in which bw_{net} is replaced by a function of the device pair, not a constant. This is a v2 priority because every fleet-scale walkthrough loses fidelity without it.

The MLIR bridge is manual. Section 4 establishes the structural correspondence to MLIR via DAG rewrite rules (Table 4) and side-by-side `linalg/scf` code (Listings 1 to 5), but the constraint checker stops at parsing molecular formulas and emitting text diagnostics. The next step—and the most impactful engineering direction we see—is to emit MLIR lowering passes programmatically through PDL pattern descriptors. A student would type a molecular formula, see the constraint diagnosis, and observe the corresponding MLIR graph transformation execute end-to-end. Closing this loop turns the framework from a whiteboard tool into a compiler artifact.

Element stability requires governance. The current 90 elements reflect the present state of ML systems. New technologies—CXL memory expanders, optical interconnects, state-space models, and whatever follows—will require revision. The irreducibility criterion (Equation (1)) provides a principled admission mechanism (a candidate enters the table only if its cost cannot be derived from same-layer composition), and Exercise 3 in Section 8.6 walks students through applying it to CXL. But the table itself is a versioned artifact, and the project will need a lightweight governance process—something like an RFC mechanism—to admit new elements without diluting the criterion. We see this as a community process that the published table seeds rather than something the paper itself can resolve.

9.3 Empirical Limitations

The third category is the one that will most affect the paper's reception.

Deployment evidence is qualitative. Section 8.8 reports three patterns observed across three semesters of teaching CS249r at Harvard. These observations have shaped how we revise the framework, but they

are not a controlled study. We cannot yet say that students taught with the framework outperform students taught without it on any external metric. A randomized study comparing cohorts is planned for Fall 2026; until it runs, the empirical case for the framework rests on the five derivation walkthroughs, not on student outcomes. We are explicit about this distinction because the difference between "the framework can derive correct answers" and "students learn to derive correct answers using the framework" is the difference between a research artifact and a curriculum.

9.4 Conclusion

Modern ML infrastructure demands a shift from memorizing branded techniques to reasoning from physical constraints, and that shift requires both a structured design space and a search procedure over it. This paper provides both halves. The Periodic Table of Machine Learning Systems (Section 2) is the design space, with 90 elements admitted by a formal Irreducibility Criterion (Section 2.3). The Constraint-Driven Lowering Heuristic (Section 3.4) is the search procedure, with four filters that prune the table to a small set of candidate interventions for any constraint violation. Molecular ML (Section 3) provides the notation in which the search operates, with separated structural and concurrency operators ($\otimes$ and $\|$) and Roofline-grounded cost rules. The Iron Laws (Section 8.1) collapse the cost calculus into three manipulable formulas. The structural correspondence to MLIR (Section 4), expressed as DAG rewrite rules (Table 4), connects the framework to the compiler infrastructure the industry is already building. And the generalized constraint operator $|_{C,\,p>\theta}$ (Section 7) makes the framework's boundary conditions explicit: deterministic within a node, probabilistic at fleet scale. None of these pieces stands alone. The Periodic Table without the heuristic is a taxonomy. The heuristic without the table has nothing to search over. The cost rules without either are just Roofline by another name. What the paper contributes is the integration, and the five walkthroughs are the evidence that the integration works.

The framework does not replace simulators, compilers, or profilers; it provides the conceptual layer above them—the whiteboard on which cross-layer design decisions are made before any code is written. The tools below it answer "how fast does this run?"; the framework answers "what should we build, and why?" Both questions matter, and the field has had good answers to the first for decades. The argument of this paper is that we now have the structural machinery to give a principled answer to the second.

References

Krste Asanovic et al. The landscape of parallel computing research: A view from berkeley. Technical Report UCB/EECS-2006-183, EECS Department, University of California, Berkeley, 2006.

Tri Dao et al. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems*, 2022.

Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

Mark Horowitz. 1.1 computing's energy problem (and what we can do about it). *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pages 10–14, 2014.

Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefler. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems*, 2021.

Norman P Jouppi et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*, pages 1–12, 2017.

Woosuk Kwon et al. Efficient memory management for large language model serving with pagedattention. *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.

Chris Lattner et al. MLIR: Scaling compiler infrastructure for domain specific computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 2–14, 2021.

Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889*, 2023.

Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kasber, and Bryan Catanzaro. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *Proceedings of the International Conference*

for High Performance Computing, Networking, Storage and Analysis, 2021.

Samyam Rajbhandari et al. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.

Vijay Janapa Reddi. MLSysIM: First-principles infrastructure modeling for machine learning systems, 2026a. URL https://mlsysbook.ai/mlsysim.

Vijay Janapa Reddi. StaffML: A physics-grounded interview question bank for machine learning systems engineers, 2026b. URL https://staffml.ai.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. In *arXiv preprint arXiv:1909.08053*, 2019.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022.